



PROYECTO FIN DE GRADO

Grado en ingeniería de computadores

Segmentación de nubes de puntos y
fusión de datos en el ámbito de los
vehículos autónomos

Autor:

Alfredo Valle Barrio

Director:

José Eugenio Naranjo Hernández

Curso 2016-2017

Índice

Resumen	7
Abstract	9
Introducción	11
Estado del arte	13
<i>Historia de los vehículos autónomos</i>	13
La patente que cambio el mundo	13
Torpedo aéreo de Sperry (Primer piloto automático)	13
UAV (Vehículo aéreo no tripulado)	14
Seguridad pasiva en los vehículos	14
Seguridad activa en los vehículos	14
Lanzamiento del primer vehículo autónomo	15
<i>Tecnologías</i>	15
LIDAR	16
Cámara	18
Cámara estéreo	19
Radar	20
GPS	20
IMU	21
Ultrasonido	22
<i>Procesamiento de la información</i>	22
Fusión de sensores	22
Inferencia Bayesiana	23
Auto localización (SLAM)	23
Detección y evaluación de obstáculos	24
Planificación de trayectorias	25
Evasión de obstáculos	27
Toma de decisiones	27
<i>Conducción autónoma</i>	28
Niveles de clasificación de coches autónomos	28
Nivel 0	28
Nivel 1	29
Nivel 2	29
Nivel 3	29
Nivel 4	29
Nivel 5	29
Sistemas autónomos actuales	29
Vehículos semiautónomos actuales	30
Mercedes Benz	30
Waymo, Google	30
Uber	31
Tesla	32
Baidu	34
Previsión del desarrollo de los vehículos autónomos	34
Metodología	37

<i>Esquema de tiempos del proyecto</i>	<i>37</i>
<i>Lenguaje de programación</i>	<i>37</i>
<i>Entornos de desarrollo y herramientas</i>	<i>38</i>
Borland C++	38
Visual Studio	38
RTMaps	39
<i>Control de versiones</i>	<i>39</i>
<i>Objetivos y metodología de trabajo</i>	<i>39</i>
Fase 1: Desarrollo de un algoritmo de reconocimiento de obstáculos para un láser lidar de 1 canal	40
Objetivo	40
Metodología de trabajo	40
Adquisición de datos	40
Resultado esperado	41
Fase 2: Desarrollo de un sistema de segmentación de puntos, reconocimiento y tracking de obstáculos y visualización de datos, para los datos provenientes de un láser LIDAR de 16 canales	41
Objetivo	41
Metodología de trabajo	44
Adquisición de datos	44
Tratamiento de datos	45
Resultado esperado	45
Fase 3: Desarrollo de un sistema de fusión de datos para obstáculos identificados desde un láser LIDAR y una cámara Mobileye	45
Objetivo	45
Metodología de trabajo	45
Adquisición de datos	46
Tratamiento de datos	48
Resultado esperado	48
Trabajo realizado	49
<i>Fase 1: Desarrollo de un algoritmo de reconocimiento de obstáculos para un láser lidar de 1 canal ...</i>	<i>49</i>
Configuración y comunicación	49
Adquisición de datos	50
Comunicación con el ordenador de bajo nivel	50
Análisis de datos	51
Segmentación	51
Tracking	53
Respuestas del algoritmo	55
Frenada de emergencia	55
Ajuste de velocidad	56
Continuación de la circulación	56
Adelantamiento	56
<i>Fase 2: Sistema de detección de obstáculos con un láser scanner de 16 canales</i>	<i>57</i>
Arquitectura	57
Interfaz grafica	58
Capa de control y memoria compartida	59
Arrays	59
Flags	60
Objetos	60
Lector de datos	61

Analizador	61
Parámetros	62
Constructor	63
Análisis	64
Flags de control	66
Flag de análisis	66
Flag de warning	66
Flag de pausa	66
Flag de tratamiento	66
Inicialización y actualización	67
Núcleo del análisis	67
Comprobar bloqueo	68
Segmentación	69
Puntos cercanos	70
Puntos cercanos en horizontal	70
Puntos cercanos en diagonal	71
Puntos cercanos en vertical	71
Filtrado y preparación de obstáculos	72
Filtrado	72
Preparación	73
Tracking	73
Relacionar por velocidad	73
Relacionar por posición	74
Copiar obstáculos	74
Representación en OpenGL	75
Constructor y parámetros	76
Modificar puntos	79
Modificar obstáculos	79
Teclas especiales	79
Interfaz de pruebas Off-line	79
<i>Fase 3: Desarrollo de un sistema de fusión de datos</i>	<i>80</i>
Problemas encontrados	81
Sincronización	81
Asociaciones	81
Ambigüedades	81
Estimación	82
Soluciones técnicas	82
Adaptador del frame rate (Pre proceso)	82
Graficas de los resultados obtenidos	83
Asociación, Matching	85
Matching_1	86
Matching_2	88
Algoritmo MHT	88
Algoritmo de puntuación	88
Comparación de similitud entre dos parámetros	89
Ambigüedades	90
Estimación	91
Teoría Bayesiana	91
Resultados	93
Aclaración de los módulos desarrollados	93
Conclusiones	95

Trabajo futuro	97
Impacto social y ambiental	99
Responsabilidad ética y profesional	101
Referencias	103
Índice de Figuras	105
Índice de ecuaciones.....	107

Resumen

Palabras clave: Lidar, Fusión de datos, conducción autónoma, Visión artificial, OpenGL.

Los vehículos autónomos ya no forman parte del futuro, si no que sus primeras implementaciones comienzan a ser visibles en la sociedad actual. Ya hay prototipos totalmente funcionales recorriendo las calles, únicamente restringidos por las leyes y el pensamiento de la sociedad actual.

Este proyecto corresponde al inicio del desarrollo del sistema de detección y evaluación de obstáculos y toma de decisiones de un vehículo autónomo. En el cual se desarrollan 2 modulo principales en el ámbito de un vehículo autónomo. La adquisición y análisis de datos de un sensor, un láser scanner Lidar, y la fusión de datos para datos recibidos de una cámara Mobileye y el láser mencionado anteriormente.

Para el desarrollo de esta memoria, se ha comenzado por la exposición del estado del arte en materia de vehículos autónomos, con el objetivo de mostrar la tecnología actual y su aplicación en el mundo contemporáneo.

A continuación, se expone la metodología de trabajo realizado, mostrando las decisiones de ingeniería que han llevado al desarrollo de este proyecto sobre el lenguaje C++ y Visual C++. Así como, las decisiones en cuanto a algorítmica y segmentación de puntos tomadas.

Como parte principal, se desarrolla el trabajo realizado dividido en 3 fases.

- Desarrollo de un algoritmo de reconocimiento de obstáculos y toma de decisiones, a partir de los datos recibidos de un láser scanner de 1 canal.
- Desarrollo de un sistema de análisis de nubes de puntos y toma de decisiones a partir de los datos recibidos de un láser scanner de 16 canales.
- Desarrollo de un sistema de fusión de datos para datos que provienen de un láser scanner de 16 canales y una cámara Mobileye.

Las tres fases tienen como objetivo principal su ejecución en tiempo real.

Por último, se exponen las conclusiones extraídas del trabajo realizado y las diferentes líneas de trabajo futuro propuesto.

Abstract

Keywords: Lidar, data fusion, autonomous driving, artificial vision, OpenGL

Data analysis and data fusion constitute the core of the intelligence of an autonomous vehicle and it is one of the most developed phases in the field of autonomous driving.

There are many techniques and algorithms for the approximation of this problem.

This project presents a solution for the implementation of an object recognition algorithm for 1 channel laser scanner. It also discusses the implementation of a complete system for point cloud analysis, obstacle recognition and decision making for the data provided by 16 channel laser scanner. Finally, a possible implementation of a data fusion system, for data that comes from a 16 channel laser scanner and a Mobileye camera is shown.

The result obtained from the developed systems are explained in the memory and it shows the result of the segmentation and analysis of the number of the point cloud in real time and the improvement in the precision and reliability of the result data from the fusion system.

Introducción

En la sociedad actual el concepto de vehículo autónomo tiene una connotación futurista, al igual que ha pasado con todas las tecnologías en sus inicios antes de su implantación en masa. Pero este futuro está más cerca de lo que la sociedad piensa. En la actualidad ya existen grandes módulos de los sistemas embebidos en vehículos autónomos que se encuentran totalmente capacitados de funcionar de manera autónoma. Como la frenada de emergencia, el control de crucero adaptativo o el sistema para mantener el vehículo en el carril.

En los últimos años, la carrera por alcanzar un vehículo autónomo de nivel 5, un vehículo autónomo que no requiere supervisión humana y es capaz de enfrentarse a situaciones complejas de conducción de manera autónoma, ha tenido un gran auge. Esto se debe a que todas las empresas punteras en el sector de la automoción han creado sus respectivos departamentos de investigación en este campo. E incluso, grandes empresas tecnológicas, no dedicadas a este sector, se han sumado a la carrera. Tanto para el desarrollo de un vehículo autónomo completo, como para el desarrollo individual de los módulos que lo componen.

Esta carrera por alcanzar un vehículo autónomo de nivel 5 tiene dos fines claramente definidos. Por un lado la búsqueda a mejorar la tecnología actual impulsada por el auge del desarrollo de los vehículos eléctricos. Y por otro lado el objetivo de incrementar la seguridad en materia de conducción, un tema que es de especial interés en la sociedad actual debido al gran número de muertos en las carreteras.

El objetivo de este proyecto es el desarrollo de un sistema de segmentación de unas nubes de puntos proporcionadas por un láser scanner y la detección de obstáculos en la misma. Como continuación se ha establecido un segundo objetivo, el desarrollo de un sistema de fusión de datos con el fin de mejorar la credibilidad de los datos obtenidos.

Para el alcance de este objetivo se ha desarrollado este proyecto en el cual se han desarrollado dos módulos de un sistema autónomo, ambos orientados al análisis de los datos proporcionados por los sensores. Ya que estos datos son la única fuente de información que tiene el sistema para conocer el entorno que lo rodea y tomar las decisiones necesarias para preservar la seguridad del vehículo.

El primer módulo consiste en el análisis de datos que provienen de un láser scanner lidar. La tecnología lidar, “light detection and ranging”, es una tecnología basada en laser mediante la cual se dispara un haz de láser y se recoge se rebote contra una superficie. Mediante esta tecnología se pueden realizar mediciones de distancia al objeto contra el que ha rebotado e incluso identificar el material a partir de la reflectividad, valor que se consigue gracias a la intensidad del rebote. Un láser scanner es un dispositivo que utiliza 1 láser o la combinación de varios, los cuales rotan proporcionando una imagen del entorno que lo rodea.

El desarrollo de este módulo se ha dividido en dos fases.

La primera fase ha consistido en el desarrollo de un algoritmo de segmentación de puntos y reconocimiento de obstáculos, para los datos obtenidos de un láser scanner de 1 canal. Gracias a que los resultados de esta fase resultaron favorables, se optó por continuar en esta línea de investigación con la siguiente fase.

En la segunda fase se desarrolló un sistema completo de segmentación de nubes de puntos y reconocimiento de obstáculos, para los datos obtenidos de un sensor Velodyne VLP-16, un láser scanner de 16 canales. Con este nuevo sensor, se ha podido obtener una imagen del entorno en 3 dimensiones lo que ha mejorado considerablemente el reconocimiento de obstáculos, a costa de la necesidad de desarrollar un algoritmo más complejo en donde la cantidad de información de entrada ha aumentado de manera considerable.

Con el desarrollo de este módulo, que fue desarrollado en Visual C++, se consiguió un sistema capaz de reconocer obstáculos en la vía y tomar decisiones para preservar la seguridad en la conducción en tiempo real.

La primera fase fue montada sobre un vehículo y se realizaron pruebas de funcionamiento en tiempo real en un entorno controlado, la pista de pruebas del INSIA.

El segundo módulo ha consistido en el desarrollo de un sistema de fusión de datos para los datos proporcionados por un láser scanner de 16 canales y una cámara Mobileye, lo que significa que recoge los datos del láser y la cámara y realiza un algoritmo de fusión de datos cuyo resultado es un set de datos más precisos y creíbles que los datos de entrada. Esto es posible gracias a la teoría de inferencia Bayesiana muy utilizada en este ámbito de sistemas autónomos. Su principal exponente es el filtro de Kalman que se puede encontrar implementado en prácticamente todos los sistemas de este tipo, gracias a sus múltiples aplicaciones.

En este último módulo se ha trabajado sobre una grabación de datos de ambos sensores, con el objetivo de que en un futuro se desarrolle un sistema de reconocimiento de obstáculos para los datos proporcionados por una cámara u otro sensor ya que este módulo fácilmente adaptable.

En el desarrollo de esta memoria se expone el estado del arte en materia de vehículos autónomos, la metodología y decisiones de ingeniería tomadas en este proyecto, el trabajo realizado y las conclusiones y los resultados que se han obtenido de este proyecto.

Por último, se exponen las líneas de investigación propuestas para la continuación de este proyecto hacia su objetivo final, el desarrollo de un sistema autónomo completo.

Estado del arte

En este apartado se van a abordar los cambios y avances que han sufrido las tecnologías que actualmente se utilizan en los vehículos autónomos.

Historia de los vehículos autónomos

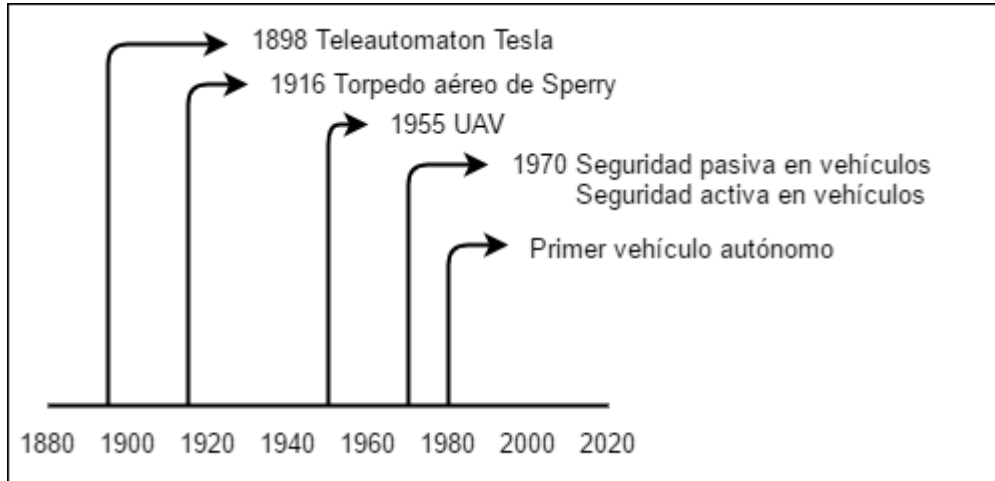


Figura 1: Línea de tiempo

La patente que cambio el mundo

El Teleautomaton de Nikola Tesla (1856-1943), es considerado el primer dispositivo a control remoto. Este dispositivo era un pequeño submarino controlado mediante ondas de radio, mediante las cuales se abrían o cerraban circuitos eléctricos en el interior del submarino, que suponían las instrucciones básicas.

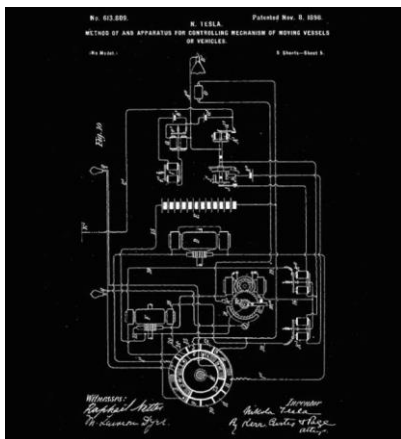


Figura 2: Teleautomaton de Tesla

En noviembre de 1898, la Oficina de Patentes concedió la patente de su dispositivo a Tesla.

Torpedo aéreo de Sperry (Primer piloto automático)

El torpedo aéreo de Sperry, también conocido como bomba volante, es considerado el primer piloto automático. El sistema consiste en un giroscopio estabilizador a través del cual se mantiene un avión volando en línea recta durante una determinada distancia

La primera demostración fue realizada en 1916 para demostrar su viabilidad.

UAV (Vehículo aéreo no tripulado)

Los UAV o vehículos aéreos no tripulados son la evolución del torpedo aéreo de Sperry. Estos dispositivos son capaces de mantener de manera autónoma un nivel de vuelo controlado y sostenido, y propulsado por un motor.

Para que una aeronave sea considerada como UAV debe ser capaz de realizar de forma autónoma tanto el vuelo sostenido como el despegue y aterrizaje.

En la actualidad existe una gran variedad de modelos de distintas formas y tamaños. Y son operados principalmente en el ámbito militar. A continuación se puede ver la imagen de un UAV MQ-9 Reaper.



Figura 3: UAV

Seguridad pasiva en los vehículos

En los inicios de los años 70 se comenzó a tomar conciencia de la seguridad en el ámbito de la automoción. Durante estos años, concretamente en 1974 entró en vigor la normativa de Tráfico que obliga el uso del cinturón. A partir de ese momento cada vez son más las normativas que se han desarrollado y los cambios que han sufrido las antiguas normativas, para la protección del ocupante del vehículo.

Seguridad activa en los vehículos

La seguridad activa en los vehículos la componen todos aquellos elementos que aumentan la estabilidad del vehículo en movimiento y, en la medida de lo posible, evitan accidentes.

Uno de los componentes de seguridad activa más conocidos es el ABS, sistema antibloqueo de frenos. Este dispositivo reduce la distancia de frenado manteniendo la capacidad de cambiar de dirección sin bloquear las ruedas.

La primera aparición de este dispositivo fue en el año 1970 cuando Bosch desarrollo una primera versión.

Lanzamiento del primer vehículo autónomo

Los primeros atisbos de un vehículo autónomo fueron en el año 1980 gracias a una furgoneta guiada por visión artificial de Mercedes-Benz. Esta furgoneta fue diseñada por un equipo de la Universidad de Múnich que condujo hasta a 100 km/h en calles sin tráfico.

Gracias a este proyecto, en la que la comisión europea mostro interés e invirtió 800 millones de euros, arrancó el proyecto EUREKA Prometeus cuyo objetivo era el desarrollo de un vehículo autónomo.

En ese momento se inició la carrera para el desarrollo de un vehículo autónomo en la que estamos sumergidos en la actualidad.



Figura 4: Vehículo autónomo de Google

En la imagen anterior se puede ver la imagen del vehículo autónomo de Google, una de las empresas más avanzadas en el sector de los vehículos autónomos.

Tecnologías

En este apartado se expondrán las tecnologías que se encuentran habitualmente en un vehículo autónomo actual, así como su cometido y su potencial en el sistema. En un vehículo autónomo se pueden encontrar gran cantidad de sensores, esto se debe a que estos son el único medio de conocer el entorno que lo rodea. Por lo que de ellos depende la seguridad del sistema durante la conducción.

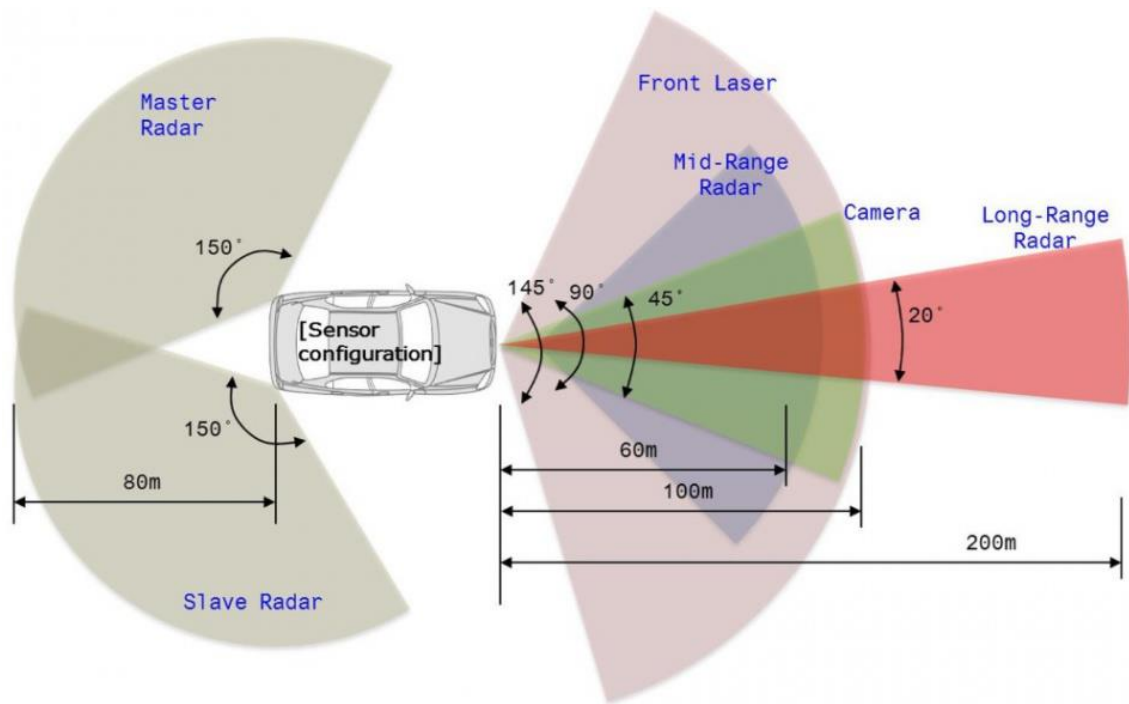


Figura 5: Estructura de sensores en un vehículo autónomo

En la imagen anterior se puede observar el esquema de sensores general de un coche autónomo. En él se pueden encontrar todos los sensores que van a ser tratados en detalle a continuación.

En la lista siguiente se indica la correspondencia de cada sensor específico con cada área de interés en la imagen.

- Lidar (Front Laser)
- Cámara (Camera)
- Cámara estéreo (Camera/Mid-Range Radar)
- Radar (Long-Range Radar/Master-Slave Radar)
- GPS (Interno)
- IMU (Interno)
- Ultrasonido (Master-Slave Radar)

LIDAR



Figura 6: Sensores LIDAR

LIDAR siglas en inglés de “Laser Imaging Detection and Ranging”, es un dispositivo habitualmente conocido como láser scanner, que usa un haz pulsado de láser para medir distancias.

Un láser scanner Lidar es capaz de medir con precisión distancias de hasta 100 metros aproximadamente, pudiendo variar dependiendo del modelo.

De él se obtiene una nube de puntos tridimensionales con unas coordenadas en un sistema de coordenadas local respecto al láser en 360 grados y con una amplitud vertical que varía dependiendo del modelo. En el Velodyne Lidar de 16 canales, el más extendido, se puede encontrar una amplitud vertical de 30 grados.

Consta de uno o varios canales cada uno de los cuales es un láser que gira y realiza una medición a una frecuencia fija.

Este tipo de sensor se encuentra en prácticamente todos los coches autónomos actuales siendo muy útiles para el mapeo del entorno, así como para detección de obstáculos. También han sido desarrollados múltiples algoritmos de auto localización (SLAM) donde se ha demostrado su gran utilidad, a pesar de la gran cantidad de datos a manejar.

En la imagen siguiente se puede ver la imagen devuelta por un Lidar HDL-E de 64 canales. Se puede observar su gran resolución y distinguir de manera rápida los obstáculos del fondo. En la imagen se pueden distinguir distintos colores en la nube de puntos debido a que el Lidar también devuelve el valor de intensidad del punto en que ha rebotado, permitiendo realizar una segmentación de la nube de puntos por colores. Además es capaz de devolver dos impactos si el objeto es relativamente transparente con lo que podemos obtener una medición tanto del objeto como del fondo.

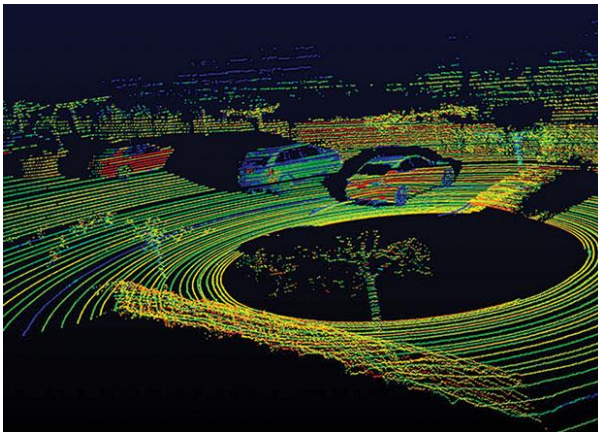


Figura 7: Visión sensor LIDAR

Cámara



Figura 8: Cámara Mobileye

Se usan cámaras de propósito general para realizar trabajos de segmentación, localización y detección de señales. Pero uno de los modelos más extendidos es la cámara desarrollada por Mobileye.

La cámara Mobileye, como la que se puede ver en la imagen de anterior, es capaz de detectar vehículos, bicicletas y peatones, aparte de hacer tracking de los mismos, pudiendo obtener su velocidad y trayectoria. También es capaz de reconocer señales de tráfico.

La cámara se conecta a la centralita del vehículo a través del CAN Bus, con lo que tiene acceso a todos los parámetros del vehículo tales como la velocidad y dirección, que le permite realizar un análisis de riesgos de cada objeto de la calzada. Todo esto es realizado mediante un procesamiento interno con lo que no se ha de realizar un post proceso de la imagen con el consiguiente ahorro en tiempo de reacción.

De ella se obtiene una evaluación del entorno, esto se puede observar en la imagen siguiente donde se pueden ver los datos obtenidos de la cámara sobre una imagen sincronizada de una cámara de propósito general.



Figura 9: Visión de una cámara Mobileye

Se puede observar que reconoce varios vehículos en la carretera y su riesgo en la conducción, se puede observar también la línea azul que representa la trayectoria

seguida por el vehículo de enfrente. También ha detectado la bicicleta y varios peatones.

Se puede ver que detecta los vehículos aparcados e indica que no suponen un riesgo para la conducción marcándolos en verde.

Cámara estéreo



Figura 10: Cámara estéreo

Una cámara estéreo es un dispositivo con dos o más lentes las cuales recoge una imagen por cada lente permitiéndole simular la visión binocular humana. Esto le permite tener una visión 3D con lo que puede hacer mediciones de distancia y profundidad.

El procesamiento de la imagen de este tipo de cámaras es similar a la de las cámaras normales como la cámara Mobileye, de la que se ha hablado anteriormente. Pero con el potencial que le da poder medir distancias mediante triangulación entre las imágenes de las dos lentes. Tal como es percibida la profundidad por los humanos.

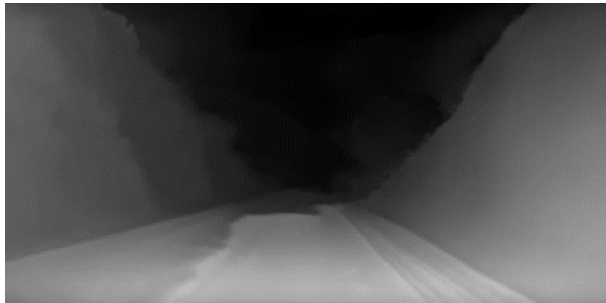


Figura 11: Visión de una cámara estéreo

En la imagen anterior se puede ver la imagen que se obtiene de una cámara estéreo, concretamente la cámara ZED. Se puede observar que se obtiene una imagen en escala de grises, los tonos más claros representan puntos más cercanos en cambio los tonos más oscuros representan los puntos más alejados.

El verdadero potencial en el uso de esta cámara está en que se unifica la capacidad de un láser y una cámara. En cuanto al láser obtenemos la capacidad de medir distancias como se ha comentado antes, y de la cámara tenemos la resolución de puntos, mucho mayor a la de un láser scanner. Con esto también se consigue simplificar el procesamiento, ya que el procesamiento de los planos de profundidad con una cámara normal con algoritmos como el Z-Buffer es muy costoso.

Radar



Figura 12: Radar

Los radares son otro de los sensores más usados en los coches autónomos, estos dispositivos utilizan ondas de radio para medir distancias.

A diferencia de los sensores LIDAR que trabajan con pulsos de luz, los radares trabajan en EHF, Extremely High Frequency, esta se corresponde a la banda de 30 a 300 gigahercios, definida por la Unión Internacional de Telecomunicaciones.

Estos tienen dos cometidos principales según su configuración. Pueden ser usados para medir a largas distancias, más allá del alcance de un sensor LIDAR o para detectar objetos en las proximidades. Se suelen usar para detectar obstáculos en los puntos ciegos del vehículo o distancias pequeñas, menores a 10 metros en las partes frontal y posterior del vehículo.

Se suelen usar debido a su bajo coste y pequeño tamaño en comparación con los sensores LIDAR.

GPS



Figura 13: GPS

El sistema GPS o Sistema de Posicionamiento Global, consiste en una red de satélites, desarrollada e instalada por el Departamento de Defensa de Estados Unidos, que

permite a cualquier dispositivo con un chip GPS geo localizarse de manera relativamente precisa.

Existen otros sistemas de posicionamiento menos extendidos como el sistema GLONASS de la federación rusa, o Galileo desarrollado por la Unión Europea. Estos sistemas están en auge, concretamente el sistema GLONASS que promete una mayor precisión, por otra parte el sistema Galileo está centrando su desarrollo en mejorar la precisión en zonas polares.

En la actualidad cualquier dispositivo electrónico cuenta con un chip GPS. La tecnología actual permite el desarrollo de sistemas GPS de precisión submétrica a un coste asequible. Es un sistema básico en cualquier vehículo autónomo, permite una localización relativamente precisa a pesar de su latencia. Un vehículo puede funcionar de manera relativamente autónoma con solo un GPS teniendo previamente mapeada la carretera y en un entorno controlado, sin obstáculos.

A parte de su uso común suele ser usado como sistema común sincronizador entre los distintos sensores, ya que en su funcionamiento el sistema GPS sincroniza su reloj con el de los satélites los cuales están sincronizados con el reloj atómico. Esto permite a todo el sistema poder estar sincronizado de manera aislada, en cuanto a conexión a internet.

IMU

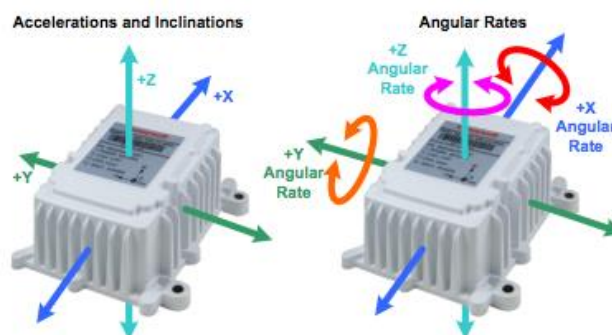


Figura 14: Unidad de Movimiento Inercial

La Unidad de Movimiento Inercial, IMU, es un dispositivo compuesto por varios acelerómetros y giroscopios mediante los cuales es capaz de medir su velocidad y orientación. Este dispositivo usa los datos de aceleraciones obtenidas por sus sensores internos para realizar un tracking de su movimiento.

Los sensores de uso comercial son precisos en cortos periodos de tiempo pero no se deben usar durante periodos largos, debido a que tienden a acumular error. Los sensores de uso militar si pueden ser usados durante largos periodos de tiempo pero su coste es demasiado elevado, por ello se suele optar por usar otros métodos de posicionamiento como principales y reservar este tipo de sensores como método de reserva.

La Unidad de Movimiento Inercial es un sensor básico en cualquier vehículo autónomo. A pesar de no ser usada para realizar mediciones directas, se usan para realizar ajustes a las mediciones de otros sensores, el más usual es el GPS. Esto se debe a que la frecuencia de medición del GPS es mucho más lenta, lo que significa que el vehículo solo supone su posición a partir de las últimas mediciones del GPS.

Se suelen usar filtros tales como el Filtro de Kalman, para hacer más preciso el conocimiento de la posición entre las mediciones del GPS, además de que permite detectar posibles errores en la medición.

Ultrasonido



Figura 15: Sensor de ultrasonidos

Los sensores de ultrasonido son dispositivos capaces de medir la distancia a una superficie de cualquier material. El único requisito es que reboten el sonido. Estos sensores emiten un sonido en forma de pulso y miden el tiempo que tarda en regresar. Son precisos en distancias de hasta 6 metros.

En el ámbito de los vehículos autónomos se suelen emplear en las zonas ciegas del vehículo. Su uso más extendido es como ayuda al aparcamiento, midiendo la distancia al vehículo u objeto anterior y posterior. Pero también tiene otros ámbitos de uso, por ejemplo como indicador en la zona ciega del retrovisor.

Procesamiento de la información

Como se ha visto en el apartado anterior, en un vehículo autónomo se pueden encontrar gran cantidad de sensores. Estos arrojan una enorme cantidad de información por segundo, pero esta información resulta inútil si no se realiza un procesamiento adecuado la misma.

En este apartado se verán los métodos más comunes para el tratamiento de dicha información, así como algunos de los algoritmos más extendidos. También se verán las principales divisiones en cuanto a procesamiento de información que se dan en un vehículo autónomo.

Fusión de sensores

Uno de los principales campos en los que se está centrando el desarrollo en cuanto a procesamiento de información es la fusión de sensores. La fusión de sensores consiste

en procesar de manera conjunta la información que se obtiene de varios sensores. El trabajo de la fusión de sensores es el siguiente:

- Se encargar de recopilar la información proveniente de los sensores.
- Procesa la información de forma inteligente de manera que detecta posibles errores en la adquisición de datos, analiza información redundante proveniente de distintas fuente y deshecha la información innecesaria.
- Genera un nuevo set de información más completa y precisa del entorno que rodea al sistema en ese momento.

Este proceso es un parte crítica del sistema e influye directamente en la toma de decisiones que se verá posteriormente.

La mayoría de los algoritmos desarrollados para realizar este proceso están basados en la Inferencia Bayesiana que se verá a continuación, aunque también existen otros métodos basados en inteligencia artificial.

Inferencia Bayesiana

La inferencia bayesiana es un tipo de inferencia estadística que se basa en el teorema de Bayes y consiste en inferir la probabilidad de una hipótesis a partir de la probabilidad de varias observaciones. Este tipo de inferencia se aplica en fusión de sensores cuando de dos o más sensores se extrae información redundante, de la cual se puede inferir otra información más precisa o “creíble” en términos de probabilidad.

Como se ha indicado antes existen varios algoritmos basados en la inferencia bayesiana para abordar el problema de la fusión de sensores, algunos de los más comunes son los siguientes:

- Filtro de información, que consiste en eliminar información redundante y manteniendo la de mayor probabilidad.
- Filtro de partículas, que es utilizado para estimar el estado un sistema que cambia en el tiempo.
- Filtro de Kalman, que utiliza una serie de observaciones en el tiempo que tienen cierta incertidumbre y producen estimaciones de variables desconocidas que tienden a ser más precisas que aquellas basadas en una única observación.

Auto localización (SLAM)

La auto localización o SLAM por sus siglas en Inglés, es una técnica que consiste en la construcción de un mapa del entorno a medida que se mueve el vehículo, a su vez le permite localizarse en el entorno y estimar su trayectoria. El principal problema que se busca resolver con esta técnica es dar la capacidad a un vehículo de ser posicionado en un entorno desconocido y que este sea capaz de construir un mapa del entorno y localizarse dentro del mismo.

Esta técnica está directamente relacionada con la inferencia bayesiana, que se ha visto anteriormente, debido al problema de la incertidumbre generada por la inexactitud de los sensores, además de la falta de precisión en cuanto a la medición del movimiento

del vehículo. Por ello se usa la inferencia bayesiana para determinar exactamente, o al menos mejorar, la posición de los obstáculos y la localización del vehículo.

Actualmente, existen dos métodos especialmente extendidos para la implementación de esta técnica, el Filtro Extendido de Kalman y los Mapas de Ocupación de Celdillas. Estas dos implementaciones destacan por sus diferencias.

El Filtro Extendido de Kalman usa la inferencia bayesiana para definir de manera más precisa los objetos y obstáculos del entorno, haciendo más precisa la localización, pero la linealización característica del Filtro Extendido de Kalman, que es lo que lo diferencia del Filtro de Kalman, provoca que las estimaciones cambien a lo largo del tiempo y no se correspondan con sus valores reales. Como consecuencia de esto la incertidumbre en las estimaciones no se corresponde con el verdadero error cometido. Además el coste computacional es exponencial respecto al número de objetos. A pesar de todo esto es la implementación más extendida debido a que describe el entorno de manera precisa y es capaz de cerrar bucles.

Los Mapas de Ocupación de Celdillas se basan en discretizar el espacio en parcelas de tamaño fijo que se identifican como ocupadas o vacías con cierta incertidumbre. Esta implementación requiere que la posición del vehículo sea conocida, por lo que se necesita un método auxiliar que estime la posición del vehículo en cada instante. Su precisión permite que el error acumulado sea pequeño, ya que cuanto más discretizado este el espacio mayor será su precisión. Algunas de sus ventajas son: su implementación es sencilla, no requiere definir completamente los objetos, la generación de trayectorias se simplifica debido a la partición del espacio en ocupado o vacío, se puede ajustar la precisión del algoritmo a la potencia computacional disponible y puede ser extendido al espacio tridimensional de manera sencilla.

Detección y evaluación de obstáculos

La detección y evaluación de obstáculos la parte del procesamiento de la información más desarrollada en el tiempo, se han desarrollado múltiples algoritmos para implementar este paso dependiendo del sensor al que se aplique.

Para los sensores cuya información son imágenes como en el caso de cámaras y cámaras estéreo, por ejemplo, se usan métodos de procesamiento de imagen con el objetivo de diferenciar los obstáculos del fondo, también se usan para realizar la segmentación de carreteras e interpretar las señales de tráfico.

Por otra parte, para aquellos sensores que realizan mediciones de distancias como los ultrasonidos, radares y laser scanner se utilizan otros tipos de técnicas como el reconocimiento de objetos mediante algoritmos geométricos, que identifican obstáculos reconociendo su forma, por densidad de puntos como el algoritmo “Voxel grid”, que divide el espacio tridimensional en celdillas, además de otros algoritmos de segmentación de nubes de puntos.

Este paso supone una parte crítica en el sistema debido, tanto a que su resultado ha de ser lo más preciso posible para asegurar el reconocimiento de todos los potenciales

obstáculos de la carretera, como que su tiempo de ejecución ha de ser tiempo real, varios algoritmos de reconocimiento de obstáculos suelen ejecutarse a la vez e independientes. Y posteriormente se suelen realizar nuevamente, algoritmos de fusión de datos entre los resultados de diferentes algoritmos para mejorar el resultado.

En este paso también se realiza el seguimiento y reconocimiento de trayectorias, “tracking”, de los obstáculos con el fin de mejorar los algoritmos de detección e identificar el riesgo que suponen para el vehículo.

Planificación de trayectorias

La planificación de trayectorias o “path planning” son estrategias que se encuentran en los coches semiautónomos o autónomos. Estas estrategias consisten en definir la ruta que ha de seguir el vehículo.

La planificación de trayectorias se suele dividir en dos niveles:

- Planificación de la trayectoria global: Que consiste en definir la ruta que va a seguir el vehículo desde el punto inicial hasta el destino.
- Planificación de la trayectoria parcial: Que consiste en definir detalladamente cual va a ser el movimiento del vehículo.

Para abordar este problema se suele usar un GPS o un sistema de planificación de rutas similar, como sistema para planificar la trayectoria global como en la imagen siguiente, en la que se puede ver la ruta definida por Google Maps para llegar a la Escuela Técnica Superior de Ingeniería de Sistemas Informáticos desde el edificio del rectorado de la Universidad Politécnica de Madrid.



Figura 16: Ejemplo de planificación global de la trayectoria

Tras haber definido cuál va a ser la ruta a seguir, se definen unos checkpoints que dividen la ruta en segmentos, de longitud variable dependiendo de la ruta, por

ejemplo en una recta los checkpoints suelen estar más separados entre sí, aunque esto depende de la técnica utilizada. En este punto es cuando entra la planificación de trayectorias parcial, esta se encarga de definir la ruta de manera precisa en cada segmento.

En la imagen de la siguiente se puede observar como la planificación de trayectorias parcial ha definido la ruta a seguir en un segmento en el que hay una curva.

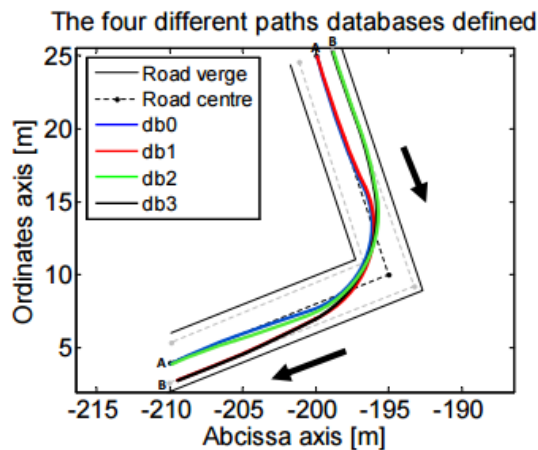


Figura 17: Aproximación de trayectoria en una curva mediante curvas de Bézier

En la imagen posterior se puede observar cómo definiría la ruta con un conjunto de curvas consecutivas, mediante la concatenación de curvas de Bézier.

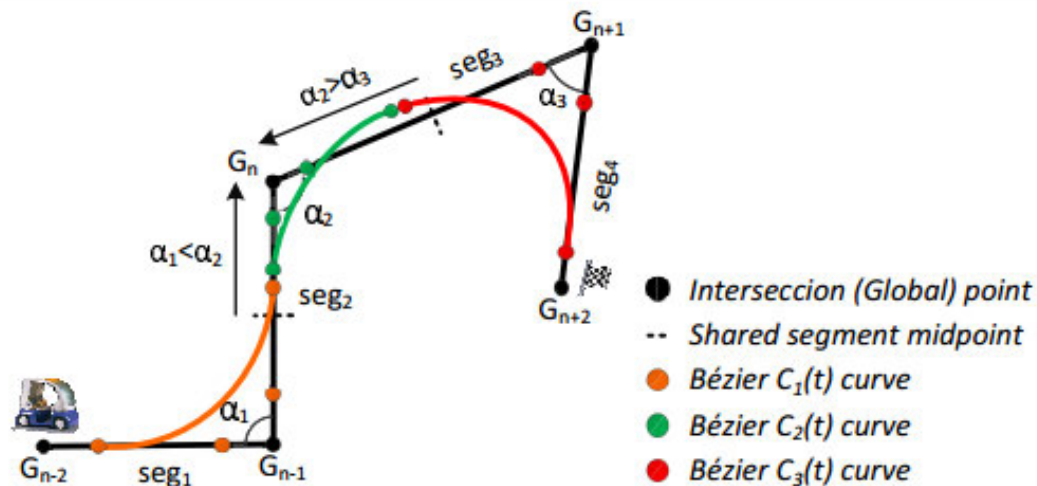


Figura 18: Aproximación de trayectoria en curvas consecutivas mediante curvas de Bézier

Hasta este punto se supone un entorno estático, lo que significa que no hay obstáculos en movimiento. El verdadero problema computacional de estas estrategias es al trabajar en un entorno de tiempo real en el que hay obstáculos móviles.

Evasión de obstáculos

En entornos reales sobre la planificación de trayectorias vista anteriormente se aplican técnicas de evasión de obstáculos, encargadas de modificar la trayectoria original con el objetivo de evitar los obstáculos que el vehículo se pueda encontrar.

Habitualmente se definen dos acciones que puede realizar un vehículo al encontrarse un obstáculo, adelantar o frenar. Existen múltiples técnicas para definir la ruta a seguir en una maniobra de adelantamiento, la mayoría están basadas en el análisis de grafos, en las imágenes siguientes se puede ver un ejemplo del resultado de la implementación de las técnicas de campos potenciales y utilizando lógica difusa.

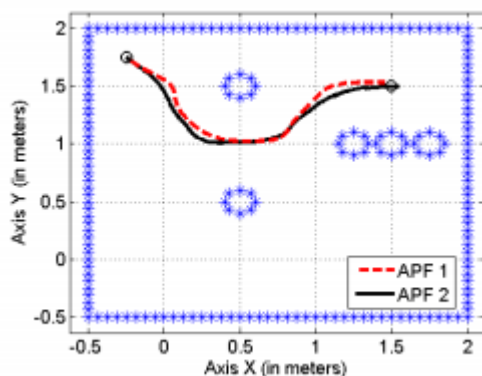


Figura 19: Evasión de obstáculos mediante campos potenciales

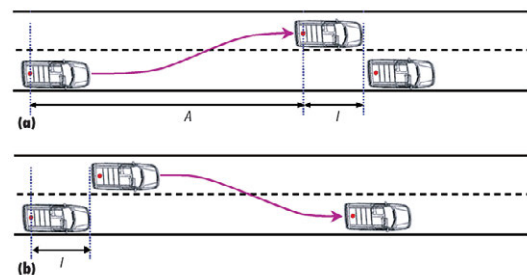


Figura 20: Aproximación de adelantamiento mediante lógica difusa

Toma de decisiones

El último paso en el procesamiento de la información en un vehículo autónomo es la toma de decisiones. Este paso no se suele ejecutar al mismo nivel que la adquisición y procesamiento de datos, si no que se da en un nivel superior, por encima de los procesamientos de datos individuales de cada sensor.

En este paso, el proceso encargado de la toma de decisiones evalúa la información obtenida del procesamiento de la información de cada sensor así como la información interna del vehículo, para actuar de la mejor manera posible dentro de las capacidades del vehículo. El sistema ha de tener en cuenta las características y limitaciones del vehículo, tales como el peso, el agarre, el ángulo máximo de giro de las ruedas, la capacidad de respuesta del motor, etcétera. A demás de lo anterior, el sistema trabaja siempre bajo unos márgenes de seguridad que aseguran la correcta respuesta del vehículo en condiciones no ideales.

Habitualmente, la toma de decisiones se encuentra fuertemente condicionada con la seguridad de las personas, tanto de los pasajeros como de los peatones u otros

vehículos de la calzada, por lo que las técnicas utilizadas para abordar este problema buscan en la medida de lo posible evitar cualquier accidente o en caso de que sea inevitable reducir al máximo los daños. Los sistemas más avanzados, en caso de accidente inevitable, identifican el tipo de obstáculos que encuentra y tratan de evitar los daños humanos por encima de todo. Recientemente, ha surgido una tendencia en cuanto a implementación de estas técnicas, iniciadas por la marca Mercedes Benz en sus vehículos, que anteponen la seguridad de los pasajeros del vehículo aun por encima de la seguridad de los peatones.

Conducción autónoma

En la actualidad, no se puede decir que exista un vehículo completamente autónomo. Por ello la Sociedad de Ingenieros de Automoción (SAE, Society of Automotive Engineers), la organización encargada del desarrollo de los estándares para vehículos, ha desarrollado una clasificación de vehículos autónomos dependiendo de la intervención humana necesaria para su circulación.

Niveles de clasificación de coches autónomos

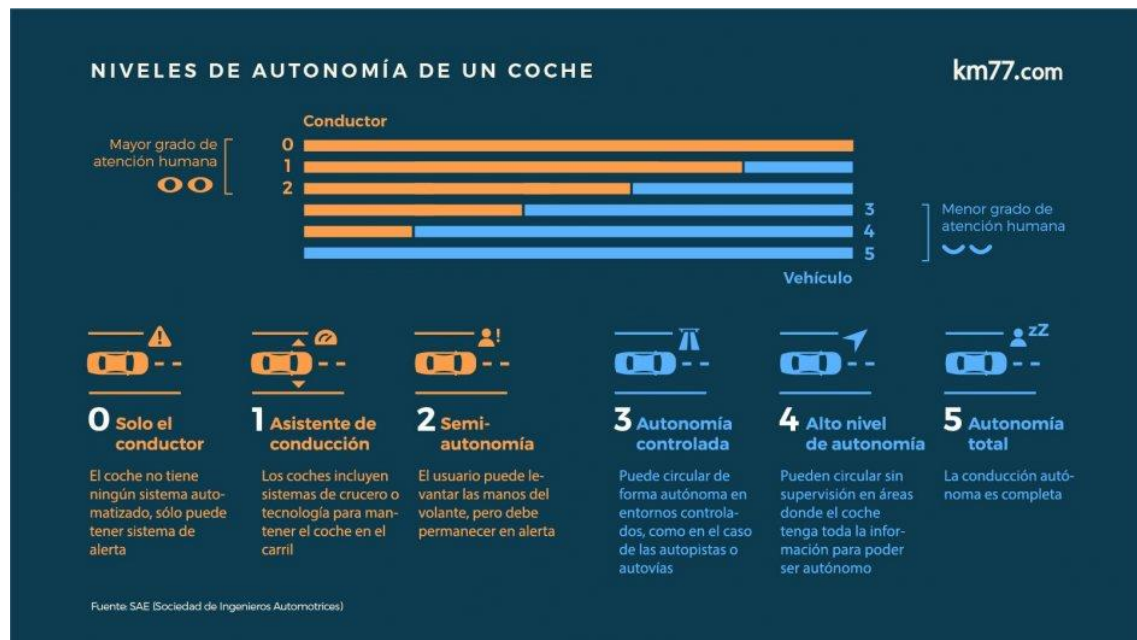


Figura 21: Niveles de autonomía de un vehículo autónomo

Los niveles de clasificación de la autonomía de un vehículo son 6 como vemos en la imagen anterior. A continuación se va a explicar en qué consiste cada nivel de autonomía de menos autónomo, nivel 0, a completamente autónomo, nivel 5.

Nivel 0

En este nivel se encuentran los vehículos que no cuentan con ningún sistema autónomo ni de ayuda a la conducción o similar, por lo que toda la responsabilidad recae sobre el conductor. Se incluyen en este nivel los vehículos con sistemas de aviso, como los sistemas de asistencia de aparcamiento o avisos de obstáculos en el ángulo ciego del retrovisor.

Nivel 1

En el nivel 1 se encuentran los vehículos con unas capacidades básicas para la conducción, y son capaces de conducir el vehículo por sí mismos. Es el caso de los vehículos que cuentan con sistemas como el control de crucero o que son capaces de mantener el coche en el carril en el que circula. Los vehículos situados en este nivel son capaces de manejar una función primaria, volante o acelerador por ejemplo, nunca más de una. En este nivel el vehículo continúa necesitando un alto grado de atención por parte del conductor.

Nivel 2

Los vehículos que se pueden encontrar en el nivel 2 reciben la denominación de semiautónomos. En este nivel el vehículo ha de ser capaz de tomar el control de la conducción, a pesar de ello el conductor ha de mantenerse atento para actuar si es necesario. Los vehículos de este nivel son capaces de manejar varias funciones primarias a la vez.

Nivel 3

En el nivel 3 se encuentran los vehículos capaces de circular en entornos controlados. A este nivel todas las funciones primarias están automatizadas permitiendo al vehículo tener libertad en la conducción. En este nivel el vehículo es capaz de tomar decisiones y realizar maniobras como adelantamientos aunque el conductor ha de permanecer atento.

Nivel 4

En el nivel 4 ya no se necesita que el conductor esté atento a la conducción. En este punto el vehículo ha de ser capaz de circular de manera totalmente autónoma y resolver cualquier situación en un entorno controlado, como entornos totalmente mapeados.

Nivel 5

En el nivel 5 se encontrarían los vehículos capaces de circular de manera totalmente autónoma en entornos desconocidos. En la actualidad no existe ningún coche que pueda clasificarse en este nivel.

Sistemas autónomos actuales

A continuación se van a exponer los vehículos autónomos que se pueden encontrar en la actualidad y sus capacidades.

Vehículos semiautónomos actuales

Mercedes Benz



Figura 22: Coche autónomo Mercedes Benz

Desde abril de 2016 está en circulación el clase E de Mercedes. Este vehículo clasificado como semiautónomo de nivel 2. El sistema, que ha de ser accionado por el conductor, mantiene al vehículo en el carril y cuenta con control de crucero. Pero no es capaz de tomar decisiones por su cuenta, por ello se encuentra en el nivel 2.

El vehículo es capaz de cambiar de carril con seguridad si el conductor se lo indica accionando el intermitente, siempre teniendo en cuenta las normas de circulación, como por ejemplo si hay línea continua no la cruzara aunque el conductor le indique el cambio de carril. A demás antes de realizar la maniobra realiza una comprobación de seguridad.

El vehículo cuenta con frenada de emergencia automática a velocidades inferiores a 60 km/h. Y velocidades superiores en caso de que un vehículo se cruce de forma transversal. Otra de las innovaciones en este vehículo es que es capaz de detectar si un vehículo se acerca por detrás a demasiada velocidad, en este caso activa las luces de emergencia para indicárselo al vehículo posterior y prepara los sistemas de seguridad tanto activos como pasivos para el impacto, precarga los cinturones, prepara el freno, etcétera.

Waymo, Google



Figura 23: Coche autónomo Waymo

Waymo es una empresa que se ha creado especialmente para la rama de vehículos autónomos de la empresa Google, debido a su madurez. Los vehículos de Waymo incorporan la tecnología driverless de Google con la que han obtenido la primera licencia de circulación para un coche autónomo en el estado de Nevada.

Estos vehículos, unos de los más avanzados hasta el momento cuentan con la tecnología necesaria y un software suficientemente avanzado para conducir de manera autónoma. Los vehículos de Waymo se clasifican en el nivel 3 en la escala de autonomía de la SAE.

En la imagen siguiente se puede ver cómo percibe el entorno el vehículo. Mediante sus laser-scanner LIDAR, sus cámaras y cámaras estéreo.

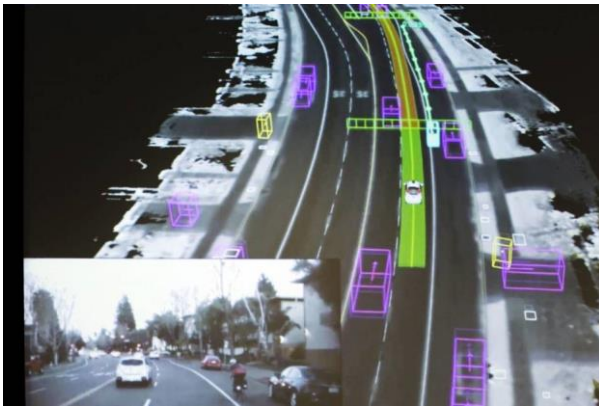


Figura 24: Visión del entorno de los vehículos Waymo

Uber



Figura 25: Coche autónomo Uber

Uber ha sido la primera empresa en probar sus vehículos en entornos reales, concretamente en Pittsburg. A pesar de que legalmente los vehículos autónomos de Uber se consideran de nivel 3 debido a que están obligados a tener siempre un conductor, tecnológicamente estarían mejor clasificados en el nivel 4 pese a aun tener fallos, como por ejemplo que no reconocen el carril bici.

En este momento, Uber es la empresa más avanzada en materia de vehículos autónomos a pesar de ser una de las últimas en meterse en este campo. Esto se debe

principalmente a su gran flota de vehículos de los que puede extraer gran cantidad de información, ya que este es su entorno de trabajo, al contrario que otras compañías que tienen que hacer grandes campañas de recolección de datos.

La previsión para los próximos años de Uber es, mediante la alianza formada a finales de 2016 con la empresa Volvo, desarrollar un paquete de hardware y software que dote al vehículo de una capacidad totalmente autónoma para circular. A continuación se puede ver una imagen del prototipo de vehículo autónomo de Uber en conjunto con Volvo.



Figura 26: Prototipo de vehículo autónomo Uber - Volvo

Tesla



Figura 27: Coche autónomo Tesla

Algunos de los modelos de vehículos de la marca Tesla incorporan un sistema denominado “autopilot”, que legalmente se considera un sistema de ayuda a la conducción, lo que significa que pertenece al nivel 2, pero realmente puede funcionar como un sistema de nivel 3 ya que es capaz de circular de manera relativamente autónoma. El sistema “autopilot” es capaz de, en entornos considerados sencillos como autopistas, mantener al vehículo en su carril, funciona como control de crucero adaptativo que mantiene la velocidad del vehículo en concordancia con la velocidad del resto de vehículos en la calzada. Además, lo que demuestra que es un sistema de nivel 3, es capaz de realizar adelantamientos automáticamente de manera segura.

Las estadísticas que han sido presentadas por la marca demuestran la seguridad del sistema a pesar de estar aun en desarrollo. Las estadísticas de accidentes por kilómetro han sido reducidas un 40% desde la incorporación del sistema en sus vehículos.

En cuanto a tecnología, los vehículos cuentan con 8 cámaras con visión de 360 grados y 12 sensores de ultrasonidos además de un radar en la parte frontal. En la imagen siguiente se puede observar como percive el entorno un vehiculo Tesla, siendo el area amarilla percivida mediante las camaras y los sensores de ultrasonido, y el area verde por el radar frontal.

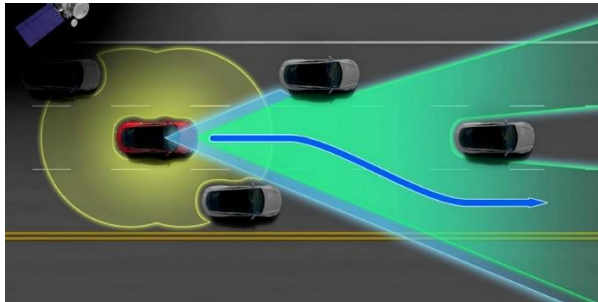


Figura 28: Percepción del entorno de un vehículo Tesla

Por último, una innovación que solo incorpora la marca Tesla es que el sistema está preparado para instalar actualizaciones automáticamente, mediante su sistema “Over-The-Air” que dota de enlace a internet al vehículo permitiendo la instalación de actualizaciones y por otra parte la recolección de datos para la empresa. Mediante este sistema la marca espera convertir, progresivamente mediante las actualizaciones, los vehículos actuales en sistemas de nivel 5.

Baidu



Figura 29: Coche autónomo Baidu

La empresa Baidu al contrario que las otras empresas que están desarrollando vehículos autónomos ha decidido no centrarse en el desarrollo de un vehículo autónomo completo si no en el desarrollo de la tecnología para modificar un vehículo convencional en autónomo.

La primera aproximación de Baidu a la construcción de un vehículo autónomo fue la modificación de un BMW Serie 3 como el que se puede ver en la imagen anterior, con el que consiguió completar un recorrido de 30 kilómetros de manera completamente autónoma y sin conductor en Pekín en 2015. Donde llegó a circular a una velocidad de hasta 100 kilómetros por hora, una velocidad bastante superior que las de otras compañías las cuales no suelen permitir la circulación de un coche completamente autónomo a más de 30 km/h y en limitados casos a 50 km/h.

Los vehículos Baidu funcionan gracias al sistema “Baidu Autobrain” que basa su funcionamiento en mapas del entorno con un alto grado de detalle, por lo que el sistema centra su potencia computacional en el reconocimiento y evaluación de obstáculos. Sus vehículos cuentan con un sensor LIDAR de 64 canales en la parte superior del vehículo, además de varias cámaras de video y radares.

Previsión del desarrollo de los vehículos autónomos

En este apartado se van a exponer las previsiones de algunas marcas en su objetivo de conseguir un vehículo autónomo.



Figura 30: Previsión de desarrollo de vehículos autónomos

En la actualidad todos los sistemas expuestos anteriormente son considerados como máximo de nivel 2, debido principalmente a las restricciones legales. Se ha expuesto la necesidad de modificar los reglamentos de circulación de los países ya que ninguno contempla la posibilidad de circulación de un coche totalmente autónomo. Los vehículos se suelen usar zonas de pruebas controladas o permisos de circulación especiales para las pruebas de circulación.

En la actualidad el sistema comercial más avanzando es considerado el sistema “autopilot” de Tesla. Y así se demuestra en su previsión de conseguir alcanzar el desarrollo de un vehículo autónomo de nivel 5 para 2018, lo cual es un objetivo muy ambicioso en comparación a los objetivos del resto de marcas que no esperan un sistema de nivel 5 antes de 2020 o 2021, incluso en algunas de las marcas más avanzadas en el sector.

En la tabla anterior podemos ver reflejadas algunas marcas de vehículos cuyos sistemas no se han presentado en el apartado anterior. Esto se debe a que estas marcas no esperan un sistema de nivel 5 si no sistemas capaces de intervenir en caso de peligro, los cuales son sistemas de niveles 3 o 4.

En general se espera que en un plazo de 5 años, alrededor de 2022 los vehículos autónomos sean una realidad. El instituto de ingenieros eléctricos y electrónicos, IEEE por sus siglas en inglés, ha estimado que para el 2040 el 75% de los vehículos serán autónomos.

Metodología

Esquema de tiempos del proyecto

El proyecto que se ha desarrollado para este trabajo comenzó en febrero de 2015 en el Instituto de Investigación del Automóvil (INSIA) junto con el profesor de la Universidad Politécnica de Madrid y director de este proyecto José Eugenio Naranjo.

El proyecto planteado en un principio, fue el desarrollo de un algoritmo de segmentación de detección de obstáculos en los datos recibidos de un láser lidar Sick LRS 1000, el cual es un láser scanner de 1 canal. El algoritmo se implementó sobre un software de adquisición de datos que había sido previamente desarrollado por el INSIA. Esta fase del proyecto fue desarrollada durante 5 meses coincidentes con mis prácticas curriculares para la universidad.

Debido a que la fase anterior obtuvo buenos resultados, se planteó la continuación del proyecto debido a la reciente adquisición de un láser scanner lidar de 16 canales. En esta fase que duró alrededor de un año, coincidente con el curso escolar 2015-2016, se desarrolló, junto a un compañero el cual se encargó de realizar el sistema de adquisición de datos y el desarrollo de la interfaz de usuario, un sistema completo de segmentación de puntos y reconocimiento y tracking de obstáculos.

Para el desarrollo de este sistema se modificó el algoritmo de segmentación de puntos desarrollado para la fase anterior de manera que realizara una segmentación de puntos en tres dimensiones para este sensor. Adicionalmente se desarrolló un sistema de representación de objetos en tiempo real mediante las librerías OpenGL. A continuación, se diseñó y desarrollo una interfaz off-line que fuera capaz de trabajar con los datos guardados del láser para poder probar los algoritmos sin tener que estar montados sobre el vehículo.

Por último, se completó este trabajo con un intership de 5 meses en el INRIA Paris (“Institut National de Recherche en Informatique et en Automatique”) concretamente en el equipo Rits, uno de los equipos punteros en investigación del automóvil, donde se desarrolló un sistema de fusión de datos encargado por la empresa Valeo. El sistema realiza una fusión inteligente de los datos de los obstáculos provenientes de un láser scanner y una cámara Mobileye.

Lenguaje de programación

El lenguaje de programación elegido para la realización de este proyecto ha sido C++, con algunas variaciones dependiendo de la etapa. Tanto en el INSIA como en el INRIA se ha decidido trabajar en C++ debido a su gran potencial en el desarrollo de sistemas de tiempo real, gracias a su velocidad de ejecución. Además se eligió este lenguaje debido a que no era necesario que fuera un lenguaje portable y siempre se desarrollaron todos los sistemas para ser ejecutados sobre el sistema operativo Windows.

En la primera etapa se utilizó C++ puro debido a que se trabajó sobre un software ya existente, que había sido desarrollado en este lenguaje.

En la segunda etapa, debido a que se desarrolló un sistema completo desde 0 se optó por utilizar Visual C++, concretamente sobre el compilador de 2015. Esto se debe al potencial que otorga Visual C++ al permitir el uso de clases manejadas, las cuales funcionan como punteros consiguiendo así mayor velocidad de ejecución y mejor manejo de memoria. Además, ya que todo el sistema se iba a desarrollar como un solo programa, y no por “módulos” en diferentes lenguajes, facilita el manejo de memoria compartida y el manejo de hilos.

En la tercera etapa, el sistema desarrollado en el INRIA se desarrolló en C++ puro pero con una estructura especial para trabajar sobre la herramienta RTMaps la cual se explicara más detalladamente en el apartado siguiente. Esto se debe a que RTMaps es la herramienta utilizada para todos los proyectos en el INRIA y sus vehículos están preparados para ejecutar los programas desarrollados en ella. Además, resultaba imperativo utilizar este formato debido a que las bases de datos proporcionadas para realizar todas las pruebas fueron proporcionadas incluidas en un módulo de esta herramienta.

Entornos de desarrollo y herramientas

En este apartado se van a exponer los entornos de desarrollo y las herramientas que han sido utilizadas para el desarrollo de este proyecto.

Borland C++

Borland C++ ha sido el entorno de desarrollo utilizado en la primera etapa del proyecto. Este entorno permite desarrollar programas en C++ para sistemas MS-DOS y Windows. Este sistema está obsoleto, su última versión es de 1999, pero ha sido utilizado debido a que el software sobre el que se implementó el algoritmo había sido desarrollado en este entorno.

Visual Studio

En la segunda y tercera etapa del proyecto se desarrolló sobre el entorno Microsoft Visual Studio 2015 ya que este entorno incorpora todas las herramientas necesarias para el desarrollo de aplicaciones en C++ y Visual C++ además de otros lenguajes de programación.

Este entorno permite depurar fácilmente el código y se puede realizar un control de versiones sobre git en el propio entorno. Otra de las ventajas es que permite crear interfaces graficas de manera sencilla, algo que resulta tedioso sin las herramientas que incorpora. Y simplifica el manejo de hilos algo que resulto de mucha utilidad debido a que el sistema se desarrolló por módulos o subsistemas que requerían una ejecución en paralelo pero con memoria compartida, la cual era manejada de manera automática por el entorno que generaba regiones críticas y semáforos internamente.

Para facilitar el desarrollo Microsoft ofrece la biblioteca MSDN (Microsoft Developer Network) que está orientada a la resolución de dudas e incluye códigos de ejemplo y documentación que simplifica el desarrollo de aplicaciones. Gracias a esto ha resultado mucho más sencillo el aprendizaje de uso de esta herramienta y el desarrollo en este lenguaje ya que era el primer contacto con ellos.

RTMaps

RTMaps es un software desarrollado por la empresa Intempora el cual es una herramienta que permite ejecutar aplicaciones en tiempo real. Esta herramienta permite desarrollar módulos los cuales se pueden desarrollar en diferentes lenguajes cuyo único requisito es seguir una estructura definida. La herramienta permite conectar dichos módulos mediante líneas en su interfaz gráfica y ella se encarga de realizar el manejo de memoria y el paso de mensajes entre los módulos cada uno de los cuales es ejecutado en paralelo, lo que simplifica su implementación.

Esta herramienta ha supuesto una gran ventaja a la hora de desarrollar el software debido a las facilidades que incorpora, a pesar de tener problemas con el manejo de memoria dinámica entre los módulos.

Los módulos implementados, han sido desarrollados sobre Visual Studio en C++, para lo cual solo ha habido que incorporarle un plugin a Visual Studio que permite crear paquetes y módulos que pueden ser leídos por RTMaps.

Control de versiones

Puesto que el sistema desarrollado era amplio y que más de una persona iba a trabajar en él, se optó por utilizar un sistema de control de versiones. El sistema de control de versiones utilizado fue git. En primer lugar porque era el sistema conocido y sencillo de manejar, y en segundo lugar porque es el sistema más extendido en el mundo laboral con lo que la experiencia obtenida de su uso sería útil en un futuro.

En un principio, en concreto para la primera etapa del proyecto se utilizó el sistema de control de versiones git sobre GitHub, pero puesto que en la segunda y tercera etapa se firmó un contrato de confidencialidad con los correspondientes centros de investigación, se optó por montar la herramienta GitLab en un servidor privado, ya que esta permite crear repositorios privados aparte de no poner un límite de los mismos, al contrario que GitHub cuyos repositorios son públicos a no ser que se compre la versión de pago y está limitado a crear 5 repositorios.

El uso del control de versiones ha resultado muy útil debido a que ha sido utilizado no solo para el control de versiones si no para llevar un control del avance del proyecto además de ayudar en la detección de errores en el código. Por último, este sistema ha resultado imprescindible ya que durante el transcurso del proyecto se ha trabajado desde distintos equipos simultáneamente debido a que han trabajado dos personas de manera conjunta en las dos primeras etapas.

Objetivos y metodología de trabajo

Como se ha visto en el esquema de tiempos del proyecto, este proyecto está dividido en tres fases:

- Fase 1: Desarrollo de un algoritmo de segmentación de puntos y reconocimiento de obstáculos para un láser LIDAR de 1 canal.

- Fase 2: Desarrollo de un sistema de segmentación de puntos, reconocimiento y tracking de obstáculos y visualización de datos, para los datos provenientes de un láser LIDAR de 16 canales.
- Fase 3: Desarrollo de un sistema de fusión de datos para obstáculos identificados desde un láser LIDAR y una cámara Mobileye.

A continuación se van a exponer detalladamente cuales han sido los objetivos del proyecto en cada fase del mismo así como las decisiones de ingeniería tomadas para su desarrollo.

Fase 1: Desarrollo de un algoritmo de reconocimiento de obstáculos para un láser lidar de 1 canal

Objetivo

El objetivo planteado en esta fase ha sido el desarrollo de una posible solución para la detección de obstáculos en la vía. El objetivo ha sido desarrollar un algoritmo en Borland C++ sobre el software ya existente, de manera que se encargara de detectar obstáculos y tomar decisiones básicas, frenada de emergencia, adaptar velocidad y adelantar. Para ello, el algoritmo se ha de comunicar con el vehículo, más concretamente con un ordenador de bajo nivel, mediante una conexión Ethernet.

El sistema completo estaría montado en un ordenador de alto nivel que recibe los datos de un láser, realiza el tratamiento de datos y comunica las decisiones al ordenador de bajo nivel mediante conexiones UDP, vía Ethernet.

Metodología de trabajo

A continuación se va a exponer la metodología de trabajo utilizada en esta fase. Definiendo el origen de los datos a tratar y el método que se ha seguido para tratarlos.

Adquisición de datos

En primer lugar la adquisición de datos.



Figura 31: Sensor LIDAR laser sick LRS 1000

Los datos a tratar son recibidos mediante puerto serie, en concreto una conexión cableada por USB a un sensor LIDAR laser Sick LRS 1000. Este sensor es un láser scanner de dos dimensiones, esto significa que devuelve una línea de puntos, mediciones de distancias. Es un láser scanner de un canal. Tiene un rango de trabajo de 0,5 a 250 metros aunque los puntos fiables para trabajar son los puntos de hasta 80

metros, y tiene un ángulo de apertura de 360 grados. Por último, este sensor puede trabajar a una frecuencia de entre 5 y 10 Hz, el objetivo ha sido que el sistema trabaje a 10 Hz.

Los datos obtenidos por este laser son recibidos de manera continua. La información que proporcionan estos datos es una distancia junto con el ángulo al que ha sido realizado el disparo.

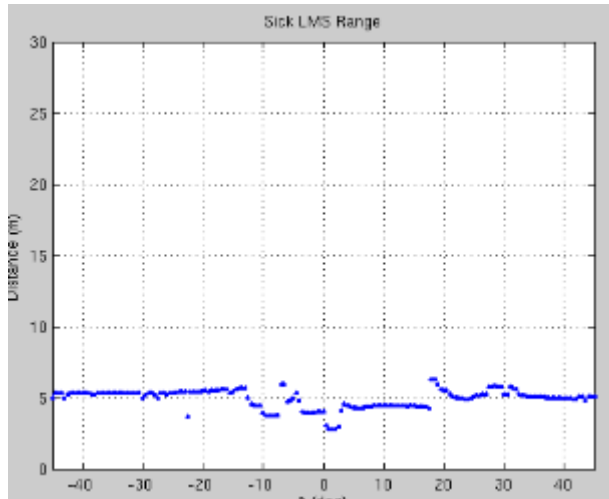


Figura 32: Ejemplo de datos provenientes del laser

En la imagen anterior se puede observar como proporciona el láser datos del entorno. En el eje horizontal esta la posición en grados del láser, en el eje vertical está representada la distancia hasta el punto donde rebotó el disparo del láser. De esta información se deben extraer los obstáculos detectados.

Resultado esperado

El resultado esperado de esta fase del proyecto mejorar el sistema existente, que era capaz de recoger la información y representarla, consiguiendo que sea capaz de detectar vehículos en cualquier parte de la calzada y sea capaz de realizar un tracking de los mismos, así como detectar cualquier objeto en la zona de riesgo, enfrente del vehículo. Tras esto el sistema deberá tomar la decisión más adecuada dentro de sus posibilidades y enviar las acciones a realizar al vehículo por medio de los sockets UDP.

Fase 2: Desarrollo de un sistema de segmentación de puntos, reconocimiento y tracking de obstáculos y visualización de datos, para los datos provenientes de un láser LIDAR de 16 canales

Objetivo

El objetivo de esta segunda fase es diseñar e implementar un sistema completo el cual se encargue de:

- Adquisición de datos de un láser scanner 3D
- Segmentación de la nube de puntos
- Detección y tracking de obstáculos
- Desarrollo de una interfaz de visualización de datos

Debido a que este sistema ya se puede considerar de un tamaño razonable y puesto que ha de respetar requisitos de tiempo real, se ha optado como hemos visto antes por su desarrollo en Visual C++. Esto se debe a que era importante poder usar las librerías y herramientas que ofrece este lenguaje, lo cual ha simplificado el desarrollo del sistema.

Para desarrollar este sistema se ha diseñado una arquitectura y un diagrama de flujo.

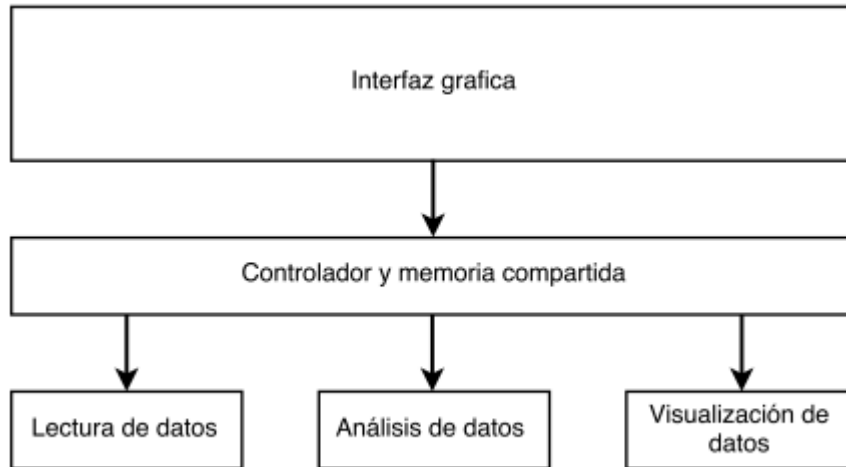


Figura 33: Arquitectura del sistema Fase 2

En la figura anterior se pueden ver los módulos principales desarrollados. Cada uno de los módulos es ejecutado, si es necesario, de manera concurrente. Se ha optado por incluir en el módulo controlador una región de memoria compartida, aunque se podría hacer en un módulo aparte, de manera que el modulo controlador maneje cualquier acceso a la misma. Además al utilizar Visual C++ se tiene la ventaja de no tener que implementar regiones críticas, semáforos o alguna herramienta similar ya que lo hace internamente. Otro de los motivos por los que se eligió Visual C++ es la facilidad que tiene para manejar el sistema operativo y crear hilos de manera sencilla. Algo que puede llegar a ser realmente complejo en otros lenguajes de programación.

Una característica especial de Visual C++ que ha sido utilizada en este proyecto es el uso de clases manejadas. Las clases manejadas se desarrollan y funcionan como clases normales, pero tienen la mejora de que internamente las funciones y todo en su interior funcionan como punteros. Esto hace que se optimice aún más si cabe en C++ el manejo de memoria y la velocidad de ejecución.

Para el desarrollo de la interfaz de visualización de datos existen dos opciones posibles para su implementación, las librerías DirectX y OpenGL. Después de investigar las ventajas y desventajas de cada una de ellas se optó por el uso de OpenGL debido a fácil implementación y su potencia, así como su compatibilidad en su uso con C++.

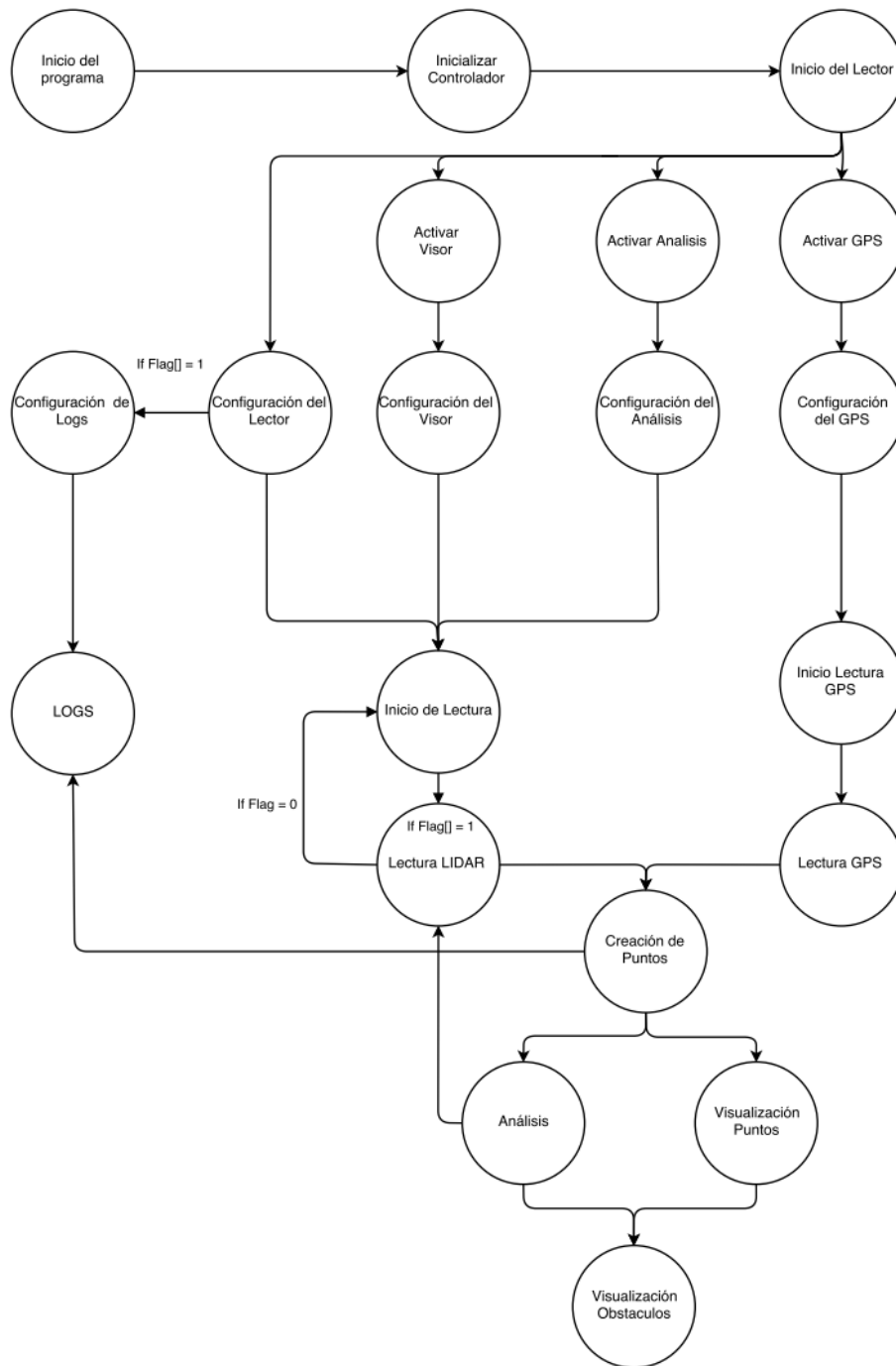


Figura 34: Diagrama UML sistema Fase 2

En el diagrama anterior se puede observar el diagrama de flujo completo diseñado para este sistema, más allá del alcance de este proyecto. Las ramas del GPS, lectura de datos y logs han sido desarrolladas paralelamente con este proyecto por un compañero en el INSIA. En el diagrama se puede observar cómo está dividido el sistema en ramas o módulos que trabajan individualmente y solo comparten información, la cual como se ha expuesto antes se mantiene en el controlador.

Metodología de trabajo

Adquisición de datos

La adquisición de datos en esta fase del proyecto ha sido desarrollada por un compañero como se ha indicado anteriormente, por lo que lo consideraremos una caja negra. Pero a continuación se detallaran las características técnicas del sensor de donde se reciben los datos, ya que es importante para el desarrollo del proyecto.



Figura 35: Velodyne VLP-16

El sensor mencionado es un Velodyne VLP-16. Este sensor es un láser escáner con las siguientes características técnicas. Es un láser scanner de 16 canales, lo que significa que consta de 16 laser situados uno encima de otro que realizan disparos simultáneamente con lo que generan una imagen 3D del entorno. El ángulo entre cada laser es de 2 grados con lo que consigue un campo de visión de +- 15 grados verticales. Cuenta con un campo de visión de 360 grados. El ángulo entre dos disparos horizontales varía dependiendo de la velocidad a la que trabaje el sensor, entre 1 y 20 Hz, lo cual se configura a través de la interfaz web que crea al encenderse. Por último, una característica muy útil es que se puede configurar el modo de disparo de manera que tiene el modo de disparo normal que devuelve la información del punto donde rebota el láser, y el modo de disparo doble que hace que si el disparo atraviesa un material transparente devuelve tanto la información del rebote al material transparente como el del rebote en lo que haya tras él.

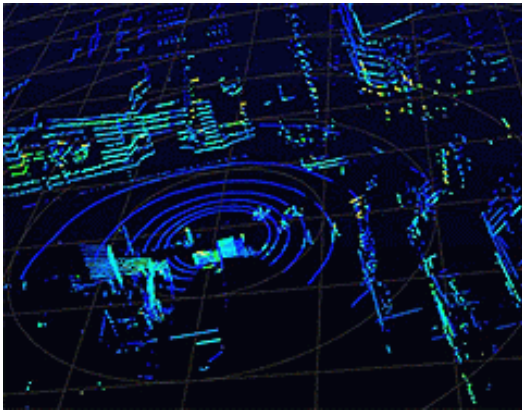


Figura 36: Representación del entorno sensor VLP-16

En la imagen anterior se puede ver un ejemplo de como este sensor percibe el entorno, como se puede observar genera una imagen tridimensional del mismo donde se pueden distinguir los objetos mucho mejor que con el sensor utilizado en la primera fase del proyecto.

Tratamiento de datos

Cabe destacar que para el tratamiento de datos como estos, con forma de nube de puntos, existen múltiples librerías tales como “Point Cloud Library”, las cuales están muy extendidas, pero tras un estudio exhaustivo de las mismas se llegó a la conclusión de que resultaban muy pesadas tanto para memoria como computacionalmente, lo que hacía peligrar los requisitos de tiempo real. Por ello se optó por desarrollar un algoritmo propio adaptado a los datos con los que se tratan, y por lo tanto más óptimo en tiempo de ejecución a partir del algoritmo desarrollado para la fase anterior.

Resultado esperado

El resultado esperado de esta fase es obtener un sistema completo modular, el cual funcione en tiempo real. Este sistema debe realizar la adquisición de datos, segmentación de puntos, detección de obstáculos, tracking de obstáculos visualización de datos y toma de decisiones. Todo ello en un lapso de tiempo de menos de 100 ms. De manera que mantenga el requisito de tiempo real sin perder ningún frame a una velocidad de 10 Hz.

Fase 3: Desarrollo de un sistema de fusión de datos para obstáculos identificados desde un láser LIDAR y una cámara Mobileye.

Objetivo

El objetivo de esta tercera y última fase del proyecto ha sido abordar otra de las etapas en cuanto a tratamiento de datos se refiere de un vehículo autónomo. Más allá de lo expuesto en las dos fases anteriores que realizaban una adquisición directa de los datos y su tratamiento, en esta fase el objetivo consiste en combinar de manera inteligente la información ya tratada proporcionada por diferentes sensores.

Metodología de trabajo

En esta fase del proyecto no se trata con datos recogidos directamente de los sensores sino con los datos almacenados en una base de datos, a pesar de ello los datos tienen

asociado un timestamp con lo que se puede simular el funcionamiento del sistema en tiempo real.

Adquisición de datos

En esta fase la adquisición de datos se realiza desde una base de datos que se introduce en un componente RTMaps. Este componente simula en entorno real enviando los datos a los componentes de tratamiento de datos como si fuera un entorno real, respetando los tiempos en los que se enviaron los datos en su grabación.

Los datos recibidos provienen de un láser lidar de 16 canales y una cámara Mobileye.

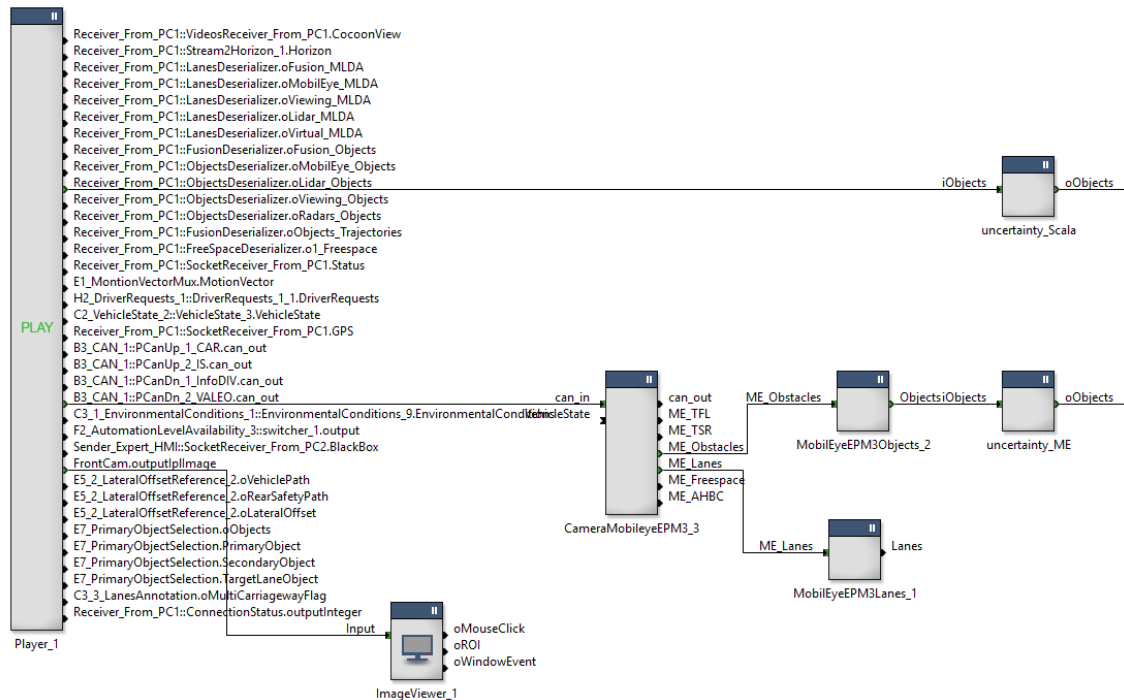


Figura 37: Componente de entrada de datos RTMaps

En la figura anterior se puede ver la estructura de componentes que se necesitan para la adquisición de datos.

En el componente Player_1 se encuentra la base de datos en la que hay una gran cantidad de información del vehículo y los sensores.

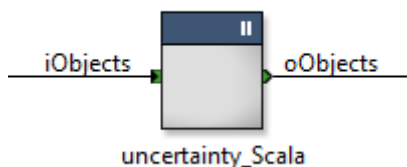


Figura 38: Corrector de posición laser

El componente uncertainty_Scala es necesario para corregir la posición de los objetos de laser ya que se encuentran en coordenadas globales.

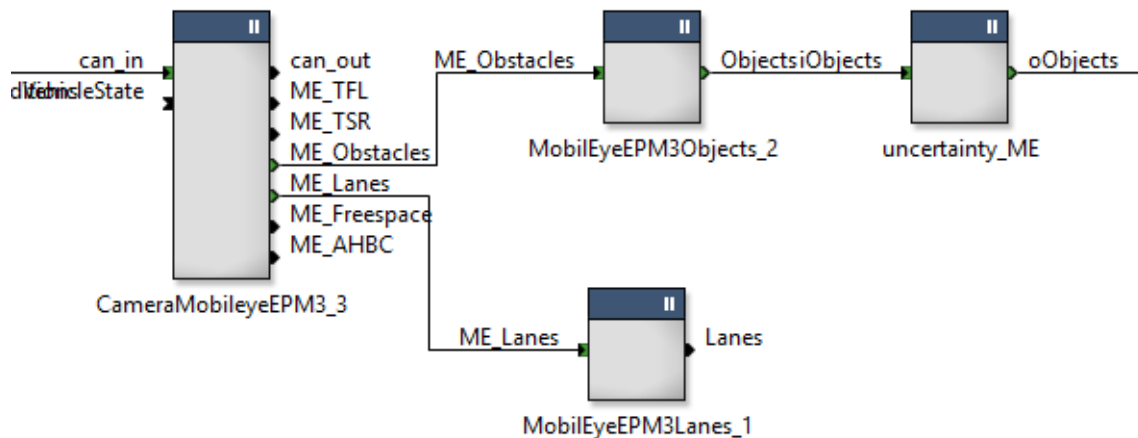


Figura 39: Adquisición de datos Mobileye

La adquisición de datos de la cámara Mobileye es un poco distinta. Para realizarla es necesario extraer la información del componente ya que el sensor proporciona mucha información que no es necesaria para este proyecto.

En primer lugar es necesario utilizar el componente CameraMobileyeEPM3_3 que es un decodificador de información y nos permite extraer toda la información proporcionada por la cámara.

Una vez se tiene toda la información disponible se extraen los obstáculos y las líneas de la carretera ya que es la información necesaria para el proyecto.

En cuanto a las líneas de la carretera son únicamente para la mejor visualización del entorno, estas requieren el componente MobilEyeEPM3_Lanes_1 para su posterior visualización.

Para los obstáculos que se utilizaran en el tratamiento de datos se necesitan dos componentes, el componente MobilEyeEPM3Objects_2 para poder tratarlos, y al igual que con los datos del láser un componente uncertainty_ME para convertir la posición de los objetos a coordenadas locales.

Los datos proporcionados por el componente son las bounding box de los obstáculos detectados. Estas bounding box han sido pre calculadas directamente por la cámara Mobileye o en el caso de los obstáculos del láser por Valeo.

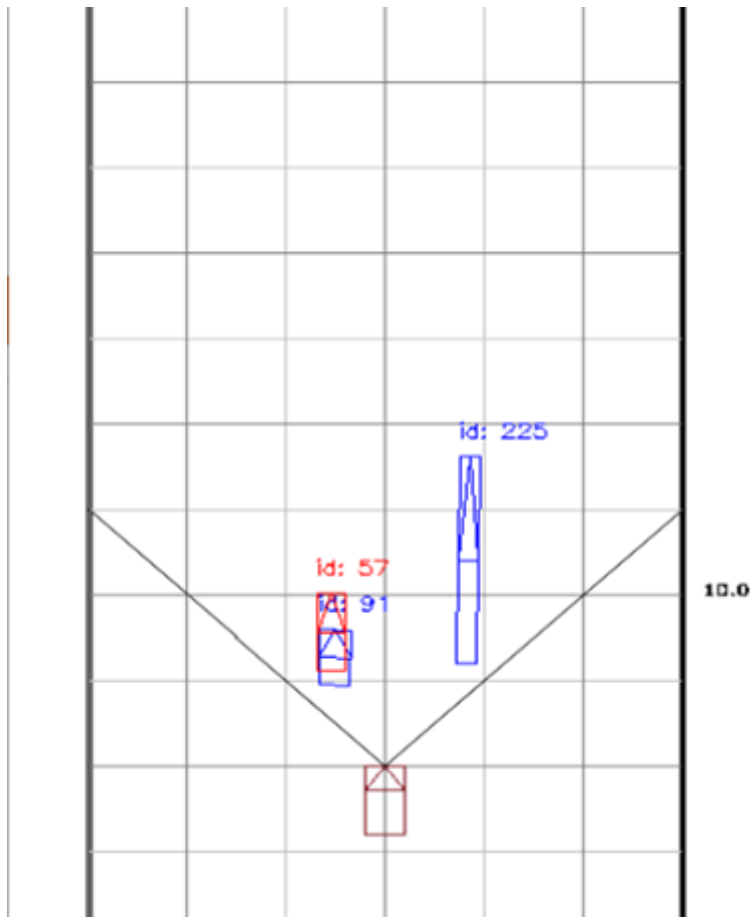


Figura 40: Entrada de datos de los sensores

En la imagen anterior se puede ver la información proporcionada por los sensores. En concreto, el objeto de color rojo es proporcionado por la cámara y los otros dos objetos, de color azul, han sido proporcionados por el láser.

Tratamiento de datos

Para el tratamiento de los datos se ha utilizado la estimación Bayesiana. Esto se puede hacer ya que los datos de los obstáculos tanto los provenientes del láser como de la cámara tienen la misma estructura y cada medición está emparejada con una probabilidad.

Gracias a esto se pueden combinar los datos de manera que la incertidumbre del dato da su peso en el resultado. Además, esta técnica permite que el sistema sea escalable pudiendo añadirse nuevos sensores sin apenas modificar el sistema, siempre que los datos recibidos de dichos sensores mantengan la misma estructura.

Resultado esperado

El resultado esperado de esta fase es, un sistema de tiempo real capaz de recibir datos de manera asíncrona de dos sensores: un láser y una cámara, y computar un resultado más fiable que la información proporcionada por los sensores de manera separada.

Trabajo realizado

Fase 1: Desarrollo de un algoritmo de reconocimiento de obstáculos para un láser lidar de 1 canal



Figura 41: Vehículo C3 con el láser scanner montado

Para la fase 1 del proyecto, se ha utilizado un láser scanner LD-LRS2100 montado en el frontal de un vehículo, Citroën C3 que se puede ver en la imagen anterior.

El sensor tiene un rango de visión de 360°, una frecuencia de exploración de hasta 10 Hz. La resolución angular del mismo varía con la velocidad de rotación.

Utiliza tecnología infrarroja de 905 nm que es segura para el ojo.

Las interfaces de conexión posibles son:

- Ethernet mediante el protocolo TCP/IP.
- Conexión RS232, puerto serie.

En este caso se ha utilizado la conexión RS232.

Configuración y comunicación

La configuración utilizada para este proyecto ha sido:

- Velocidad de rotación = 10 Hz
- Angulo de visión 60°

Una de las configuraciones más utilizadas.

Para la comunicación con el sensor se ha utilizado un software previamente desarrollado por el INSIA, mediante el cual se realiza la conexión con el sensor para su configuración, y se envían las órdenes de funcionamiento para iniciar y parar la rotación, la visualización de datos en la pantalla, y el inicio del análisis.

La parte desarrollada en este proyecto ha sido un módulo de análisis que se ha añadido al software original.

El método para la utilización del sensor es el siguiente.

En primer lugar se establece la conexión con el sensor mediante un protocolo de inicialización que configura los parámetros del láser.

Tras ello se puede iniciar la rotación a la velocidad previamente configurada e iniciar o no la visualización de datos por pantalla.

Adquisición de datos

En cuanto a la adquisición de datos, al ser utilizada la conexión mediante la interfaz RS232, puerto serie, los datos son recogidos de la siguiente manera.

El sensor envía de manera continua cada medición, la cual contiene información de la posición de rotación y distancia.

Los datos se recogen de manera directa en el módulo de análisis y se agrupan en un vector de 2 dimensiones hasta que se completa un frame para poder ser analizado, lo cual se realiza en un hilo de ejecución distinto de manera que no se pierda ningún dato en el tiempo de análisis.

Comunicación con el ordenador de bajo nivel

Antes de comenzar con la explicación del funcionamiento del análisis de datos vamos a explicar la otra parte del sistema, ordenador de bajo nivel.

El software desarrollado por el INSIA junto con el módulo de análisis están embebidos en el ordenador de alto nivel, este es el encargado de la interfaz con el usuario, y la adquisición y análisis de datos. Existe otro ordenador, el ordenador de bajo nivel que es capaz de manejar el vehículo. Este ordenador de bajo nivel controla el coche mediante el CAN Bus o mediante otras interfaces construidas por el INSIA.

Para que el resultado del análisis de datos tenga alguna repercusión sobre el vehículo se utiliza este ordenador de bajo nivel en el que se ejecuta otro software desarrollado por el INSIA también, que abre una serie de puertos TCP/IP mediante los que se le pueden dar órdenes al vehículo.

Hay más puertos abiertos, pero nos centraremos en 3.

Puerto de velocidad, mediante el cual se puede enviar la velocidad objetivo y este responderá con la velocidad actual del vehículo.

Al enviar la velocidad objetivo el vehículo ya se encarga automáticamente de alcanzar esa velocidad de manera suave mediante un controlador de lógica difusa que tiene

implementado. Con lo que no es necesario desarrollar un controlador que envíe la velocidad periódica y escalonadamente para alcanzar la velocidad objetivo de manera suave.

Puerto de volante, este puerto se utiliza para controlar el giro del volante. Se envía un ángulo de giro al puerto y, como en el caso anterior, este devolverá la posición actual del volante.

Por último el puerto de emergencia, si se ha de realizar una parada de emergencia por causas como que el sistema de alto nivel ha quedado bloqueado, se envía una señal a este puerto que indicara al vehículo que debe realizar una frenada de emergencia.

Análisis de datos

El análisis de datos se ha dividido en dos fases.

- Fase de segmentación
- Fase de tracking de obstáculos

Antes de entrar en detalles del tratamiento de obstáculos, se han de tener en cuenta algunas restricciones en los datos de entradas.

A pesar de que el sensor tiene un alcance máximo de 250 metros, los puntos fiables están comprendidos entre 1 y 80 metros.

Se establece una distancia mínima de 1 metro debido a que para los puntos en los que no se ha obtenido medición el láser indica que su distancia es 0 metros.

Y se establece una distancia máxima de 80 metros, debido a que se ha comprobado, aparte de estar indicado en el libro de especificaciones del sensor, que el rendimiento máximo del láser con reflectividad de los puntos al 10% es de 80 metros. Lo que significa que a partir de esa distancia si la superficie en la que impacta el láser tiene una reflectividad de menos del 10%, lo que está condicionado también por la luz solar y las condiciones del entorno al usar infrarrojos, es posible que el objeto no fuera detectado.

Por otra parte, se descartan directamente los puntos a más de 80 metros ya que a partir de esa distancia, y por la velocidad de análisis de datos, un objeto a más de 80 metros no supone un peligro fehaciente. Y como ventaja añadida se reduce el número de puntos haciendo más rápido el análisis de datos.

Segmentación

Una vez filtrados los puntos no válidos, es necesario agrupar los puntos en obstáculos.

Para ello se ha desarrollado un método dividido en 2 fases.

En la primera fase se agrupan los puntos, considerando que dos puntos pertenecen al mismo obstáculo si se encuentran a una distancia inferior a una tolerancia preestablecida.

Esta tolerancia viene dada por la siguiente formula que calcula la tolerancia entre dos puntos teniendo en cuenta la divergencia del haz de láser.

Ecuación 1: Ecuación de tolerancia de distancia entre 2 puntos

$$\Delta d(r_i, r_{i+1}) = S_0 + S_1 \cdot \min(r_i, r_{i+1})$$

Donde S_0 es una constante y S_1 es una constante para la reducción de ruido y como límite inferior.

Ecuación 2: Ecuación de reducción de ruido

$$S_1 = \sqrt{2 - 2 \cdot \cos(\Delta\alpha)}$$

Donde $\Delta\alpha$ representa la resolución angular.

La implementación de esta ecuación es la siguiente:

```
bool Deteccion::puntosCercanos(Punto *p1, Punto *p2)
{
    float s0 = 1.4;
    float s1 = sqrt(2 - (2*cos(2*resolucionAngular*PI/180)));
    float r = p1->getModulo();
    if (p1->getModulo() > p2->getModulo())
        r = p2->getModulo();
    float tolerancia = s0 + (s1 * r);

    return (tolerancia > p1->distanciaPunto(p2));
}
```

Figura 42: Implementación de la ecuación de tolerancia

En la imagen anterior se puede ver la implementación de la ecuación de manera que devuelve si dos puntos pertenecen al mismo obstáculo o no.

Una vez obtenidos todos los obstáculos, se vuelve a realizar un filtrado en el que se descartan los obstáculos no válidos. Aquellos que no tienen la cantidad de puntos necesaria. Esto se debe a que tras la ejecución de esta fase todos los puntos son incluidos en algún obstáculo, con lo que puede haber obstáculos con un solo punto, lo que no se considera válido dada la resolución angular del láser.

Al final de esta fase se obtiene una lista de obstáculos válidos.

Antes de pasar a la siguiente fase, tracking de obstáculos, se comprueba si existe un bloqueo. Se considera que hay un bloqueo si alguno de los obstáculos esta frente al vehículo a una distancia inferior a 5 metros.

Se comprueba de la siguiente manera:

```

bool Deteccion::comprobarBloqueo(std::vector <Obstaculo*> & obstaculos)
{
    for(int k=0; k<obstaculos.size(); k++)
    {
        if (obstaculos[k]->getCentro()->getx() > -1.5 && obstaculos[k]->getCentro()->getx() < 1.5)
        {
            if (obstaculos[k]->getMargen()<5)
            {
                return true;
            }
        }
    }
    return false;
}

```

Figura 43: Comprobar bloqueo

Como se puede ver en la imagen anterior, se comprueba si la posición del obstáculo, concretamente la coordenada x del mismo se encuentra en un valor de entre -1.5 y 1.5 lo que significa que se encuentra frente al vehículo.

Tras ello se comprueba si el margen es menor a 5 metros, siendo margen la distancia del vehículo al punto más cercano del obstáculo.

Si se detecta que existe un bloqueo no se continúa con el tratamiento de datos y se le indica al vehículo que debe realizar una frenada de emergencia.

Tracking

Para la realización del tracking de obstáculos se necesitan dos escenarios o frames consecutivos.

Para su ejecución en tiempo real se guarda el vector de obstáculos del frame anterior t_{i-1} para compararlos con los del frame actual t_i .

Una vez se tienen los obstáculos de dos frames consecutivos se recorre el vector de obstáculos del frame anterior y se busca una posible correlación con algún obstáculo del frame actual.

Pueden darse dos casos:

- Que el obstáculo si haya sido detectado anteriormente, por lo que ya se habrá estimado su dirección y velocidad.
- Que el obstáculo no haya sido detectado con anterioridad, por lo que la única información de la que dispone es de su posición y la posición de los puntos que lo componen.

En el caso de que se haya detectado anteriormente se calcula su posición prevista en el frame actual y se busca un obstáculo cuya posición corresponda aproximadamente con la posición estimada.

Debido al poco tiempo transcurrido entre dos frames y la velocidad máxima del vehículo 45 km/h no se considera necesario hacer predicciones mediante curvas, solo se considera que el obstáculo avanzara hacia delante en su dirección. En caso de que este describiendo una curva se seguirá detectando su relación mediante la tolerancia.

Una vez relacionado el obstáculo, si es posible, se actualiza la información de velocidad y dirección del mismo en el frame actual y se continúa con el siguiente obstáculo.

En el segundo caso, donde el obstáculo no ha sido detectado con anterioridad, se busca la relación en la posición y orientación del obstáculo.

De manera que la distancia entre los obstáculos sea menor a una constante previamente definida teniendo en cuenta las velocidades de circulación. Y que tengan una orientación similar.

Si se encuentra un par de obstáculos relacionables se calcula su velocidad y dirección.

```
if (matrix[j][1].getVelocidad() >= 1)
{
    if(fabs(matrix[i][0].getCentro()->distanciaPunto(matrix[j][1].getCentropred())) < 4)
    {
        relacionarVel(i,j,VCOCHE,resolucionAngular);
        indice = j;
    }
}
else if(matrix[i][0].getCentro()->distanciaPunto(matrix[j][1].getCentro()) < minimo &&
fabs(matrix[i][0].getYaw() - matrix[j][1].getYaw()) < 5)
{
    relacionarPos(i,j,VCOCHE,resolucionAngular);
    indice = j;
}
```

Figura 44: Implementación de la posible relación entre obstáculos

En la imagen anterior se puede observar la implementación del método para relacionar obstáculos por velocidad o posición según corresponda.

Por ultimo en cuanto a cálculo de parámetros de los obstáculos, para poder reaccionar ante los obstáculos que representen un peligro para el vehículo se calcula el tiempo de colisión, TTC por sus siglas en inglés.

Para realizar estos cálculos se consideran las posiciones del obstáculo y el vehículo así como sus trayectorias y velocidades.

Se considera que ambos definen un movimiento rectilíneo, por lo que se calcula la intersección de sus trayectorias.

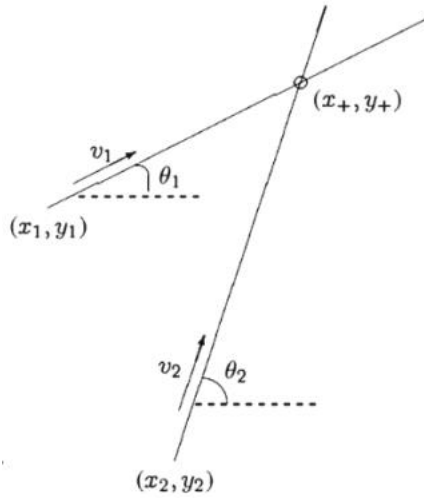


Figura 45: Intersección de trayectorias

Mediante las siguientes formulas:

Ecuación 3: Ecuaciones de la intersección de trayectorias

$$x_+ = \frac{(y_2 - y_1) - (x_2 \times \tan \theta_2 - x_1 \times \tan \theta_1)}{\tan \theta_1 - \tan \theta_2}$$

$$y_+ = \frac{(x_2 - x_1) - (y_2 \times \cot \theta_2 - y_1 \times \cot \theta_1)}{\cot \theta_1 - \cot \theta_2}$$

Una vez obtenido el punto de intersección se calcula el tiempo para la colisión.

Se define factor de seguridad, δ en el que se considera que el obstáculo y el vehículo no pueden estar en el mismo punto.

$$|T_{x_1} - T_{x_2}| < \delta$$

Donde T_{x_1} y T_{x_2} son los tiempos que tardan el obstáculo y el vehículo en alcanzar el punto de intersección (x_+, y_+) .

Respuestas del algoritmo

El algoritmo tiene 4 respuestas posibles:

- Frenar
- Ajustar la velocidad
- Continuar la circulación
- Adelantamiento

Frenada de emergencia

La activación de la frenada de emergencia puede ser producida en dos situaciones.

- Que se produzca un bloqueo, como se ha explicado antes, que haya un obstáculo en la zona de peligro. Dado que es una situación de alto riesgo no se realiza la fase de tracking para dar la respuesta lo más rápido posible.
- En caso de que haya un obstáculo en movimiento frente al vehículo, se detecte una futura colisión y un ajuste de velocidad no asegure la seguridad de

vehículo. Si durante la frenada se considera que el obstáculo ha dejado de suponer un riesgo para el vehículo se continuara circulación normal.

Ajuste de velocidad

En caso de detectar un obstáculo que pone en riesgo al vehículo pero con un TTC alto, se ajustara la velocidad del vehículo para evitar la colisión. La velocidad objetivo del vehículo será a al que este circulando el obstáculo en caso en que circule en la misma dirección con el fin de adaptar su velocidad a la del otro vehículo.

Continuación de la circulación

En caso de que no exista una situación de riesgo o que ya se haya adaptado la velocidad con el vehículo que le precede se mantendrán los parámetros de velocidad objetivo y ángulo de volante, sin ser necesario enviar la señal al ordenador de bajo nivel.

Adelantamiento

No se entrara en detalles en la maniobra de adelantamiento implementada debido a que se ha realizado por un compañero.

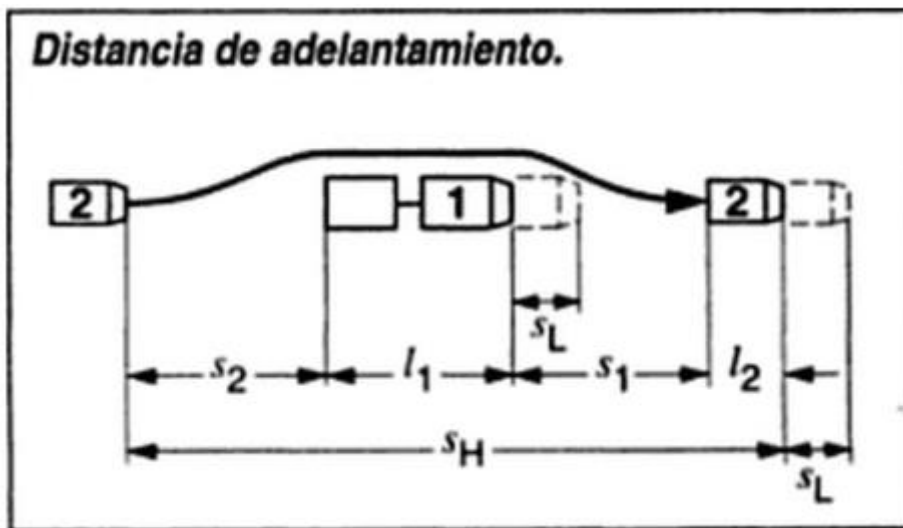


Figura 46: Distancia de adelantamiento

La maniobra de adelantamiento consiste en dos cambios de carril, teniendo en cuenta la distancia de seguridad que viene dada por la siguiente formula.

Ecuación 4: Ecuación de la distancia seguridad

$$S_x = 0,5 \times V_i \left(\frac{km}{h} \right)$$

Para el caso propuesto:

$$S_2 = 0,5 \times V_{vehiculo} \left(\frac{km}{h} \right)$$

$$S_1 = 0,5 \times V_{obstaculo} \left(\frac{km}{h} \right)$$

El cambio de carril se divide en 3 giros del volante controlados por un cálculo del tiempo necesario para cada uno.

Se tiene en cuenta el tiempo necesario para la maniobra completa de adelantamiento, la cual debe cumplir las normas de circulación.

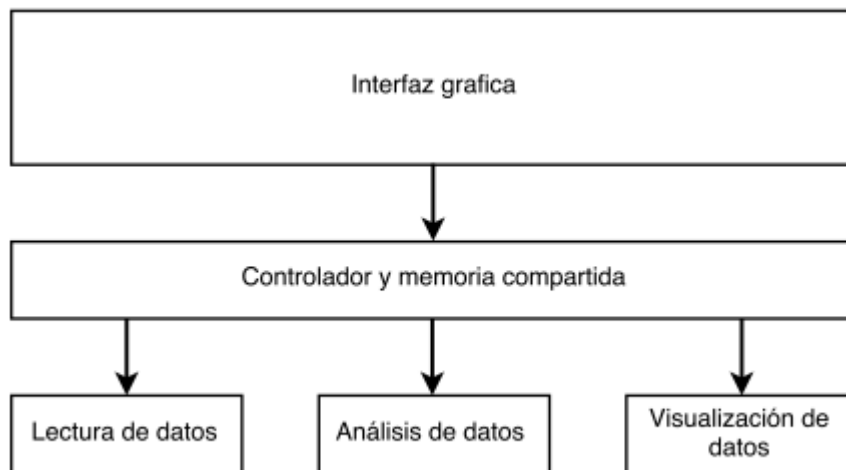
Fase 2: Sistema de detección de obstáculos con un láser scanner de 16 canales

Para esta segunda fase del proyecto se ha desarrollado un sistema de detección de obstáculos a partir de la información recibida de un láser scanner de 16 canales. Concretamente el sensor VLP-16.

En primer lugar se va a exponer la arquitectura definida para este proyecto:

Arquitectura

Ecuación 5: Arquitectura básica Fase 2



La arquitectura principal del sistema es la que se puede observar en la imagen anterior.

El sistema está montado con una interfaz gráfica que es la que inicia el funcionamiento del sistema mediante una señal en el controlador.

Cada uno de los componentes que se ven en la imagen se ejecuta en un hilo de ejecución separado, esto se ha desarrollado así con el objetivo de paralelizar el trabajo. Ya que si todo se ejecutara en un hilo, excepto la interfaz gráfica y la visualización de datos que deben ejecutarse en un hilo separado obligatoriamente, el sistema se colapsaría en muy poco tiempo. Esto se debe al volumen de trabajo, ya que el láser emite alrededor de 300.000 puntos por segundo. Además, sería imposible cumplir el requisito principal, la ejecución en tiempo real.

A continuación se va a explicar el funcionamiento de cada uno de los componentes por separado y sus comunicaciones.

Interfaz grafica

En primer lugar la interfaz gráfica, que es la base de todo el sistema. Hay que explicar que los componentes no arrancan a la vez que la interfaz gráfica, es esta la que envía las señales correspondientes para activar cada uno de los componentes, los cuales son totalmente independientes unos de otros.



Figura 47: Interfaz grafica

No se va a entrar en detalle del desarrollo de la interfaz gráfica, esta sección es únicamente para mostrar su estructura y las partes que la componen.

La interfaz gráfica está dividida en varias secciones, pero la principal y más importante es la columna situada a la izquierda.

En esta columna se encuentran todos los controles del sistema. En primer lugar, en la parte más alta el puerto de conexión, que puede ser modificado en la interfaz de control del láser. No se ha incluido la dirección ip ya que está definida estáticamente en el código fuente.

Una vez definido el puerto de conexión, se pulsa el botón de conexión y aparecerán los controles de sistema.

El primero el lector, el cual no ha sido desarrollado en este proyecto, y será tratado como una fuente directa de datos, cuenta con una serie de parámetros para ajustar la posición e inclinación del láser en el vehículo, así como el rango de detección. Los controles del rango de detección se incluyen y son importantes para el proyecto ya que facilitan el funcionamiento del análisis, ya que hace un filtrado previo al mismo eliminando los puntos fuera de ese rango.

A continuación el análisis, parte principal de este proyecto. El sistema puede funcionar con este módulo apagado para poder hacer visualización de datos directa o guardar logs para su posterior análisis.

Tras ello, otro modulo que no se ha desarrollado en el ámbito de este proyecto, el GPS. Sea ha incluido al final en el sistema, para utilizarlo como marca de tiempo y para posibles proyectos futuros.

Por último el visor, el cual no requiere de parámetros y que tiene su interfaz gráfica separada. Esto se debe a que tiene el objetivo de ejecutarse en su propia pantalla.

El resto de la interfaz gráfica, los rectángulos blancos de la parte derecha son diferentes consolas utilizadas para el seguimiento del funcionamiento del sistema y para alertas. Ya que este sistema, actualmente, no tiene acción directa sobre el vehículo. Se ha pensado como un sistema de aviso.

Todos los cambios en los controles explicados anteriormente tienen una acción directa en la capa de control y memoria compartida que se expondrá a continuación.

Capa de control y memoria compartida

La capa de control y memoria compartida es la encargada de manejar cuando debe realizar su tratamiento cada módulo, el lector, el análisis y el visualizador. Y también detecta bloqueos en los threads.

Para hacer esto cuenta con 3 estructuras de memoria principales que se expondrán a continuación.

Arrays

Son los que contienen los parámetros de funcionamiento necesarios para la ejecución del sistema.

```
void Controller::InitArrays()
{
    ArrayDataAnalisys = gnew cli::array<Object^>(SIZE_ARRAY_ANALISYS);
    ArrayDataReader = gnew cli::array<Object^>(SIZE_ARRAY_READER);
    ArrayGps = gnew cli::array<Object^>(SIZE_ARRAY_GPS);
    Threads = gnew cli::array<Thread^>(NUMERO_THREADS);
    for (int i = 0; i < SIZE_ARRAY_ANALISYS - 1; i++)
    {
        ArrayDataAnalisys[i] = safe_cast<System::Object^>(0);
    }
}
```

Figura 48: Inicialización de Arrays del controlador

Como se puede ver en la imagen anterior hay 4 Arrays de control.

Por orden son:

- Array de análisis. Contiene 7 parámetros que se extraen de otras partes del sistema según su configuración.

- Array del lector: Contiene 16 parámetros que se extraen principalmente de la interfaz gráfica además de otros que se usan para la comunicación entre threads.
- Array del GPS: Contiene parámetros de tipo de trama GPS
- Array de threads: Contiene los identificadores de los threads, se utilizan principalmente para el destructor al apagar el sistema y para comprobar el estado del mismo.

Flags

El array de Flags contiene una serie de valores booleanos, mediante los cuales se maneja el funcionamiento del sistema.

También contiene los indicadores de que módulos están o no funcionando, y se utiliza para el control de tiempo real.

```
//##### FLAGS #####
#define SIZE_ARRAY_FLAGS 7

#define FLAG_TRATAMIENTO 0
#define FLAG_LOG 1
#define FLAG_WARNING 2
#define FLAG_PAUSA 3
#define FLAG_ANALISYS 4
#define FLAG_OPENGL 5
#define FLAG_ANALISYS_MURO 6
```

Figura 49: Array de flags

En la imagen anterior se pueden ver los flags que se utilizan para el control del sistema.

Objetos

Por último los arrays de objetos que son 2 principales y 1 para el resultado del análisis.

```

void Controller::InitObjects()
{
    Obstaculos = gcnew List<Obstaculo^>(NUMBER_OBSTACLES);
    Puntos = gcnew List<Punto3D^>(NUMBER_OF_POINTS);
    Conclusiones = gcnew List<double>();
    Conclusiones->Add(0);
    Conclusiones->Add(0);
}

```

Figura 50: Inicialización de objetos del controlador

Como se puede ver en la imagen anterior los dos arrays principales son el array de obstáculos los cuales están limitados a 250 obstáculos posibles. Y el array de puntos el cual está limitado a 15000 puntos, esta cantidad ha sido calculada a partir de las especificaciones del sensor y comprobada empíricamente. Corresponde al número máximo de puntos en un frame.

Estos arrays son donde se guarda la información de salida de los módulos para poder comunicarse entre ellos. Y mediante los flags del apartado anterior se controla su acceso.

Por último, el array de conclusiones que tiene 2 posiciones en las cuales se almacena la velocidad objetivo y el ángulo de volante para el caso en que el sistema, en un futuro, tenga comunicación directa con el vehículo.

Lector de datos

El lector de datos no se ha desarrollado en el ámbito de este proyecto y por ello no se entrara en profundidad en su explicación.

El sensor proporciona dos puertos de conexión a los que se puede conectar mediante una interfaz web que tiene incluida. Mediante esta interfaz se pueden manejar múltiples parámetros del sensor como la velocidad o resolución. En el otro puerto donde no está situada la interfaz se encuentra la lectura de datos.

El sistema se conecta mediante una conexión TCP/IP como hemos visto en el apartado de la interfaz gráfica. Mediante la cual el sensor envía paquetes del tipo UDP con una estructura definida en sus especificaciones.

Este componente realiza una escucha continua en este puerto y reorganiza los paquetes con el objetivo de componer los frames. Una vez el componente ha generado un frame entero crea una lista de puntos que es enviada a la capa de control para que el resto de componentes pueda acceder a ella.

Analizador

El analizador, el componente principal de esta fase del proyecto, es el encargado de realizar el análisis de datos y extraer conclusiones.

En primer lugar se van a exponer los parámetros necesarios para su funcionamiento, los cuales están contenidos en el array de análisis visto en la capa de control.

Parámetros

Los parámetros necesarios para el análisis son los siguientes.

```
//##### ANÁLISIS #####
#define SIZE_ARRAY_ANALISYS 7

//Parametros
#define HORIZONTAL_RESOLUTION 0
#define VERTICAL_RESOLUTION 1
#define OPENING 2
#define CAR_VELOCITY 3
#define INFORME_ANALISYS 4
#define TOLERANCE 5
#define FRECUENCIA 6

//Constantes
#define ALTURA_MINIMA_OBST 0.2
#define VERTICAL_TOLERANCE 1200
#define DIAGONAL_TOLERANCE 500
#define HORIZONTAL_TOLERANCE 500
#define NUMERO_FILAS 16 //Coincide con el numero de canales
#define TAMAÑO_MINIMO_OBSTACULO 20
#define TIEMPO_MARGEN 0.3 //TODO: Ajustar parametro
#define DISTANCIA_MAXIMA 4
#define PI Math::PI
```

Figura 51: Parámetros necesarios para el análisis

Los parámetros del análisis son los siguientes al mismo orden que aparecen en la imagen anterior:

- Resolución horizontal: Indica la resolución horizontal del láser ya que puede ser configurado para realizar disparos en un ángulo concreto y no en los 360º posibles.
- Resolución vertical: De la misma manera que el anterior la resolución vertical del sensor puede ser configurada mediante la interfaz web.
- Apertura: Es el ángulo de interés de lectura en grados. Esto significa que a pesar de contar con una resolución horizontal más amplia se puede restringir el análisis de datos a un área concreta, en el caso por ejemplo de trabajar con un log.
- La velocidad del vehículo para el cálculo de velocidad de los obstáculos.
- El informe análisis contiene un puntero a la consola de la interfaz gráfica reservada para el análisis con el objetivo de que pueda mandar mensajes a la interfaz.

- Tolerancia: Factor de multiplicación de tolerancia que se maneja desde la interfaz gráfica.
- Frecuencia: Es necesario indicar la frecuencia de rotación del láser mediante la interfaz gráfica para los cálculos para la segmentación de la nube de puntos.

Además de los parámetros son necesarias una serie de constantes para el algoritmo. Las cuales son las tolerancias de distancia para la segmentación de la nube de puntos y algunas restricciones para el filtrado tanto de puntos como de los obstáculos generados. Aparte de la constante matemática PI que se utiliza para los cálculos.

Constructor

A continuación se expone el constructor utilizado para la ejecución del análisis.

```
DataAnalysis::DataAnalysis(List<Punto3D>^ puntosController, List<Obstaculo>^ ObstaculosController,
cli::array<Object>^ ParamAnalysis, List<double>^ Conclusiones, cli::array<bool>^ Flags,
cli::array<Thread>^ Threads, OpenGL^ Dibujador)
{
    try {
        //Carga de Los punteros de comunicacion con la capa de control y memoria compartida
        this->Informe = Informe;
        this->Threads = Threads;
        this->Flags = Flags;
        parametros = ParamAnalysis;
        parametros[INFORME_ANALISIS] = "";
        this->Conclusiones = Conclusiones;
        //Inicializacion de objetos
        matriz_pointer = puntosController;
        matriz = gcnew List<Punto3D>(NUMBER_OF_POINTS);
        ObstaculosvAnt = ObstaculosController;
        Obstaculos = gcnew List<Obstaculo>(NUMBER_OF_OBSTACLES);
        Cercanos = gcnew cli::array<bool>(4);
        PCercanos = gcnew cli::array<Punto3D>(4);
        //Arranque del thread de analisis
        if (!thread_analysis) {
            thread_analysis = gcnew Thread(gcnew ThreadStart(this, &DataAnalysis::Esperar));
        }
        thread_analysis->Start();
        //Guarda el identificador de thread en el array de threads del Controllerador
        Threads[1] = thread_analysis;
    }
    //Control de errores
    catch (Exception^ e) {
        Informar(e->ToString());
    }
}
```

Figura 52: Constructor del análisis

Como se puede observar en la imagen anterior, el constructor del análisis está dividido en 3 fases.

En primer lugar, se guardan de manera local los punteros a los arrays de información contenidos en la capa de control y memoria compartida, los cuales son:

- Informe: Puntero a la consola de la interfaz gráfica.
- Threads: Al arrancar el Thread de análisis, se guarda su identificador en el array de threads array para finalizar la ejecución correctamente.
- Flags: Necesario para el control de tiempos y acceso a memoria
- Parámetros: Es una lista de tipo objeto que contiene toda la información necesaria para la ejecución del análisis.

- Conclusiones: Puntero al array de conclusiones para enviar el resultado del análisis a la capa superior.

Tras ello se inicializan los arrays de puntos y obstáculos, los cuales se guardan localmente para que el resto del sistema los pueda seguir utilizando y reducir los problemas de acceso a memoria.

A continuación, se realizan las inicializaciones de objetos necesarias para la primera ejecución del análisis ya que este requiere de dos frames para funcionar.

Por último, se realiza la inicialización del Thread análisis mediante funciones incluidas en el lenguaje, Visual C++. El thread se inicia en una función llamada Esperar, en la cual el thread se mantiene al finalizar su ejecución a la espera de que el lector genere un nuevo frame.

En el constructor, se realiza un control de errores con el objetivo de que si sucediera algún error no bloqueara al sistema, si no que informa a la capa superior, para que esta tome las medidas necesarias, a su vez que lo indica mediante la interfaz gráfica.

Análisis

En esta etapa se va a desarrollar el algoritmo utilizado para la segmentación de la nube de puntos y reconocimiento de objetos.

La ejecución principal del algoritmo se lleva a cabo en la función AnalisisThread () de la cual se verá una imagen general a continuación y tras ello se entrara en detalle de cada una de sus fases de ejecución.


```

void DataAnalysis::AnalysisThread()
{
    Informar("Iniciando Thread Analisis");
    while (Flags[FLAG_ANALISYS] && !Flags[FLAG_WARNING] && !Flags[FLAG_PAUSA])
    {
        if (!Flags[FLAG_TRATAMIENTO]) {
            Informar("Tratamiento");
            matriz->AddRange(matriz_pointer);
            resolutionH = Convert::ToDouble(parametros[HORIZONTAL_RESOLUTION]); //Resolucion
            resolutionV = Convert::ToDouble(parametros[VERTICAL_RESOLUTION]); //Resolucion
            VCOCHE = Convert::ToDouble(parametros[CAR_VELOCITY]); //Vcoche
            apertura = Convert::ToDouble(parametros[OPENING]); //Apertura
            NUMERO_COLUMNAS = matriz->Count / NUMERO_FILAS;
            frecuencia = Convert::ToInt32(parametros[FRECUENCIA]);
            if (/*VCOCHE > 5*/true) {
                if (!comprobarBloqueo(matriz))
                {
                    Segmentacion(matriz, apertura);
                    if (Obstaculos->Count > 0) {
                        EliminarObstaculos(); //Elimina obstaculos no validos
                        prepararObstaculos(); //Geometria del obstaculo
                        RelacionarObstaculos(); //Tracking
                        //Copiar el vector de obstaculos obtenido en el
                        //controlador y comprueba colisiones
                        copiarObstaculos();
                    }
                }
                else {
                    Informar("Bloqueo");
                    consigna_velocidad = 0.0;
                }
            }
            else {
                Informar("Error Velocidad minima tratamiento");
            }
            //Fin tratamiento
            //Actualizar consignas en el vector de conclusiones
            Conclusiones[0] = consigna_velocidad;
            Conclusiones[1] = consigna_volante;
            Flags[FLAG_TRATAMIENTO] = true;
            matriz->Clear();
        }
    }
    //En caso de que se desactive y se reactive despues hay que limpiar los objetos
    ObstaculosvAnt->Clear();
    Esperar();
}

```

Para comenzar con la ejecución hay 4 flags de control que indican si se ha de realizar el análisis de los datos.

- Flag de análisis
- Flag de warning
- Flag de pausa
- Flag de tratamiento

La función de cada uno de los cuales se explicara a continuación.

Flags de control

Los flags de control manejan la ejecución del algoritmo. Mediante estos flags se detectan bloqueos en los diferentes módulos del sistema, el modulo del lector y el módulo de análisis que son los que dependen entre sí para la ejecución del sistema.

Flag de análisis

El flag de análisis indica al componente si debe estar activado o no. Este flag es necesario debido a que el componente puede ser apagado o encendido en tiempo de ejecución, de esta manera no es necesario matar o crear el thread en tiempo de ejecución y se puede mantener el estado del sistema.

Flag de warning

El flag de warning se activa cuando algún componente detecta que no se está cumpliendo el requisito de tiempo real en el sistema. Este flag puede ser activado tanto por el lector, como por el análisis.

Si este flag se activa, el sistema deja de ejecutar el análisis e indica el error por la interfaz gráfica.

Flag de pausa

Como su propio nombre indica este flag es activado desde la interfaz gráfica cuando el usuario desea parar la ejecución. Gracias a este flag la ejecución puede ser parada y reiniciada tantas veces como se quiera sin causar errores de ejecución en el algoritmo. Y sin que incurra en un error de tiempo real.

Flag de tratamiento

El flag de tratamiento indica 2 cosas. En primer lugar, el flag indica si los datos están listos para ejecutar un análisis sobre ellos. Y en segundo lugar, indica si se está cumpliendo el requisito de ejecución en tiempo real.

Ambas cosas se encuentran estrechamente relacionadas. Esto se debe a la manera de ser activado o desactivado el flag, lo cual se explica a continuación.

Lo primero indicar que significa que el flag de tratamiento se encuentre a true o false. Si el flag se encuentra en true significa que el análisis ha finalizado y está a la espera de un nuevo frame. Si el flag se encuentra en false, significa que se está ejecutando el análisis. Por lo tanto se deduce que, el lector es el encargado de poner el flag de tratamiento a false cuando un frame esté listo para ser analizado y este ya alojado en la capa de control y memoria compartida. Y por otro lado, que es el componente de análisis el que pone el flag de tratamiento a true cuando haya terminado su ejecución.

Una vez aclarado su significado y como se controla el turno de ejecución, se va a exponer como se realiza el control del requisito de tiempo real.

Este requisito se comprueba durante el análisis de cada frame debido a que el tiempo de generar un frame nuevo es relativamente estable y viene dado por la velocidad de rotación del láser. Por ejemplo a una velocidad de 10 Hz, el tiempo entre un frame y otro es 100 ms, y este es el tiempo del que dispone el análisis para ejecutarse. De esta

manera, el lector, cada vez que genera un frame nuevo comprueba si el análisis ha terminado, en caso de que este componente este activado. En caso de que no lo haya hecho significa que no cumple el requisito de tiempo real y activo el flag de warning.

Inicialización y actualización

Una vez realizado el control de ejecución se ha de hacer una inicialización en el caso del primer frame o una actualización en el resto, de los parámetros necesarios para los cálculos en la ejecución del algoritmo.

```
matriz->AddRange(matriz_pointer);  
resolutionH = Convert::ToDouble(parametros[HORIZONTAL_RESOLUTION]);  
resolutionV = Convert::ToDouble(parametros[VERTICAL_RESOLUTION]);  
VCOCHE = Convert::ToDouble(parametros[CAR_VELOCITY]);  
apertura = Convert::ToDouble(parametros[OPENING]);  
NUMERO_COLUMNS = matriz->Count / NUMERO_FILAS;  
frecuencia = Convert::ToInt32(parametros[FRECUENCIA]);
```

Figura 53: Inicialización de parámetros

En la imagen anterior se pueden ver los parámetros a actualizar.

En primer lugar se ha de reajustar el tamaño de la matriz para los puntos de este nuevo frame, ya que estos cambian en cada vuelta del láser. Esto se debe a que la velocidad de rotación del sensor no es totalmente estable.

Se ha de actualizar las resoluciones vertical y horizontal y la apertura ya que estos parámetros pueden ser cambiados en tiempo de ejecución mediante la interfaz web que proporciona el láser.

Se necesita la velocidad del coche en ese instante ya que afecta al cálculo de velocidades y trayectorias de los obstáculos.

Hay que calcular el número de columnas ya que la matriz no tiene realmente formato de matriz si no que es una lista de puntos, y es necesario saber el número de columnas para reconstruir la lista localmente en forma de matriz, lo que facilita la ejecución del algoritmo.

Por último, se actualiza la frecuencia del láser ya que afecta a los cálculos para la segmentación de la nube de puntos.

Núcleo del análisis

Una vez actualizados los parámetros es el momento del análisis de la nube de puntos. A continuación se muestra una imagen del código utilizado para ser explicado detalladamente a continuación.

```

if (/*VCOCHE > 5*/true) {
    if (!comprobarBloqueo(matriz))
    {
        Segmentacion(matriz, apertura);
        if (Obstaculos->Count > 0) {
            EliminarObstaculos();//Elimina obstaculos no validos
            prepararObstaculos();//Geometria del obstaculo
            RelacionarObstaculos();//Tracking
            //Copiar el vector de obstaculos obtenido en el
            //controlador y comprueba colisiones
            copiarObstaculos();
        }
    }
    else {
        Informar("Bloqueo");
        consigna_velocidad = 0.0;
    }
}
else {
    Informar("Error Velocidad minima tratamiento");
}
//Fin tratamiento

```

Figura 54: Núcleo del análisis

La primera restricción que se ha definido para realizar el análisis es que la velocidad del coche sea mayor a 5 km/h, esto se debe a que no se considera necesario realizar un análisis en parado ya que no se puede proporcionar ninguna respuesta útil. Por ahora, esta restricción esta anulada debido en primer lugar a que no hay comunicación directa con el vehículo, con lo que no se puede obtener la velocidad en tiempo real. Y en segundo lugar, porque el sistema está en fase de desarrollo y de esta manera no es necesario trabajar en un entorno completamente real.

Tras ello, se entra en el verdadero análisis de la nube de puntos, se comienza con una medida de seguridad que es comprobar si se está bloqueando el frente del vehículo, para lo cual se ha implementado la siguiente función.

Comprobar bloqueo

```

bool DataAnalysis::comprobarBloqueo(List<Punto3D>^ matriz)
{
    //Devuelve true cuando hay bloqueo
    int medio = (matriz->Count / 16) / 2;
    Punto3D^ prueba;

    for (int k = medio*0.95; k < medio*1.05; k++) { //5 % de vision delantera
        for (int i = 0; i < NUMERO_FILAS; i++) {
            if (matriz[convaPos(k, i)] && matriz[convaPos(k, i)]->valido && matriz[convaPos(k, i)]->getDistance() < 15
                && matriz[convaPos(k, i)]->getCoordinatesZ() > ALTURA_MINIMA_OBST) { //Altura de bloqueomayor k 0.2
                prueba = matriz[convaPos(k, i)];
                return true;
            }
        }
    }
    return false;
}

```

Figura 55: Comprobar bloqueo

En la imagen anterior se puede ver el código fuente de la función de comprobar bloqueo. En la cual puesto que el ángulo 0 se encuentra mirando a la parte trasera del vehículo, en vez de buscar la posición del ángulo 180 se coge directamente el punto medio de la matriz, y se comprueba un 2,5% del ángulo total a cada lado del punto medio.

Se comprueba:

- Si el punto existe
- Si el punto es válido, si no ha sido filtrado por el lector
- Si su distancia es menor a 15 metros
- Y si su altura es mayor a 0,2 metros

Si cumple todas las características anteriores significa que ese punto pertenece a un obstáculo que está obstruyendo el avance del vehículo, por lo que la función devuelve true.

En caso de que se detecte un bloqueo, el análisis no continua y se manda una consigna de velocidad igual a 0 km/h. Y se informa a través de la interfaz gráfica.

Segmentación

Una vez comprobado que no hay un bloqueo, se ejecuta la segmentación de la nube de puntos.

El algoritmo de segmentación de la nube de puntos utilizado es un algoritmo de los vecinos modificado y optimizado para esta estructura de nube de puntos. El proceso que sigue el algoritmo es el siguiente.

1. Recorre toda la matriz por columnas desde la columna en el ángulo 0, orientada hacia atrás en el vehículo, hasta la 360. Pero solo analiza los puntos dentro del ángulo de apertura.
2. Se inicializan los parámetros de cálculo
3. En caso de que sea el primer punto se asigna directamente al obstáculo 0 de la siguiente manera.

```
//Mete al final del vector de obstaculos un obstaculo que crea
Obstaculos->Add(gcnew Obstacle());
matrix[convaPos(columna, fila)]->setObstacle(0);
//Accede al obstaculo 0, al vector de componentes, mete el punto en el vector de componentes
Obstaculos[Obstaculos->Count - 1]->components->Add(matrix[convaPos(columna, fila)]);
```

Figura 56: Crear un nuevo obstáculo

4. Genera un nuevo obstáculo y lo añade a la lista de obstáculos. Marca el punto con el obstáculo al que pertenece. Y añade el punto a la lista de componentes de dicho obstáculo.
5. Si no es el primer punto, se comprueba si el punto está lo suficientemente cerca de los puntos que le rodean. No los 8 puntos, si no los que ya han sido tratados, el de arriba a la izquierda, el inmediatamente superior, el de arriba a la derecha y el de su izquierda. Esto permite reducir el número de cálculos

necesarios a la mitad. Como se muestra en la imagen siguiente.

✓	✓	✓
✓	P	X
X	X	X

Figura 57: Puntos a comprobar si son puntos cercanos

6. Si son puntos cercanos el punto es añadido al obstáculo al que pertenezca el punto cercano. Si no tiene ningún punto cercano se añade el punto a un obstáculo nuevo.
7. Se comprueba si tiene más de un punto cercano, en cuyo caso si los obstáculos a los que pertenecen son distintos, estos obstáculos se agrupan en uno solo. De esta manera si el laser por ejemplo no detectara una ventana, aun así generaría un obstáculo con el vehículo completo, ya que los obstáculos de la izquierda y derecha de la ventana se unirían por la parte de debajo de la misma.
8. Por último, se realiza la comprobación de si los puntos de la primer y última fila son cercanos, en caso de que el ángulo de apertura sea 360 grados.

Puntos cercanos

A continuación se va a exponer como se decide si dos puntos son cercanos o no.

Se han definido 3 tipos de comprobaciones:

- Puntos cercanos en horizontal
- Puntos cercanos en vertical
- Puntos cercanos en diagonal

Esto se debe a que no se utiliza la misma tolerancia a la distancia en los distintos puntos comprobados, explicados en el apartado anterior, ya que la resolución vertical y horizontal es diferente.

Puntos cercanos en horizontal

Para comprobar si dos puntos del mismo canal, lo que significa que están en la misma horizontal, son cercanos se utiliza el siguiente método.

```
bool DataAnalysis::puntosCercanosH(Punto3D^ p1, Punto3D^ p2)
{
    double tolerancia = p1->getDistance() * tan(resolutionH * PI / 180);
    tolerancia = tolerancia * ((100 + HORIZONTAL_TOLERANCE) / 100);
    return(tolerancia > p1->distanceToPoint(p2));
}
```

Figura 58: Código fuente de los puntos cercanos en horizontal

Esta comprobación es la más sencilla de las tres.

Se calcula la tolerancia en base a la resolución horizontal y la distancia del punto, y se añade el multiplicador de tolerancia que se ajusta manualmente desde la interfaz gráfica.

Y se comprueba si la distancia de un punto a otro, función que está definida en la clase punto, es menor a la tolerancia calculada.

Puntos cercanos en diagonal

El cálculo de si dos puntos diagonales son cercanos es algo más complejo que el anterior.

```
bool DataAnalysis::puntosCercanosD(Punto3D^ p1, Punto3D^ p2)
{
    double toleranciaV = p1->getDistance() * tan(resolutionV * PI / 180);
    double toleranciaH = p1->getDistance() * tan(resolutionH * PI / 180);
    Punto3D ^a = gcnew Punto3D(toleranciaH, toleranciaV, 0);
    double tolerancia = a->getModule();
    tolerancia = tolerancia * ((100 + DIAGONAL_TOLERANCE) / 100);
    return(tolerancia > p1->distanceToPoint(p2));
}
```

Figura 59: Código fuente de los puntos cercanos en diagonal

Como se puede ver en la imagen anterior, se calcula de la misma manera que en el caso anterior, pero se ha de calcular una tolerancia tanto vertical como horizontal.

Ambas tolerancias se incluyen en una estructura de punto por el único motivo de simplificar el código, ya que esta estructura ya incluye las funciones para el manejo de vectores.

Y se obtiene el modulo del punto, que funciona como un vector de 3 dimensiones, y lo cual coincide con la tolerancia.

Por ultimo de la misma manera que en el caso anterior se añade a la tolerancia su factor de multiplicación y se comprueba la distancia entre puntos.

Puntos cercanos en vertical

Se explica el cálculo de los puntos cercanos en vertical en último lugar debido a que no funciona como la tolerancia horizontal, sino que es más cercana a la diagonal.

```
bool DataAnalysis::puntosCercanosV(Punto3D^ p1, Punto3D^ p2)
{
    double toleranciaV = p1->getDistance() * tan(resolutionV * PI / 180);
    double toleranciaH = p1->getDistance() * tan(((resolutionH - (2 * 20 * 0.00001843 * 180)) / 16) * PI / 180);
    Punto3D ^a = gcnew Punto3D(toleranciaH, toleranciaV, 0);
    double tolerancia = a->getModule();
    //double tolerancia = p1->getDistance() * tan(resolutionV * PI / 180);
    tolerancia = tolerancia * ((100 + VERTICAL_TOLERANCE) / 100);
    return(tolerancia > p1->distanceToPoint(p2));
}
```

Figura 60: Código fuente de los puntos cercanos en vertical

Esto se debe a que existe un pequeño desfase entre el disparo de un canal y el siguiente, con lo que los puntos no están exactamente en la misma vertical. Por ello se

utiliza el mismo método que en el caso de puntos cercanos en diagonal pero añadiendo el desfase en la tolerancia horizontal.

Filtrado y preparación de obstáculos

Una vez segmentada la nube de puntos y obtenidos los obstáculos se realiza un filtrado y preparación de los obstáculos. A continuación se puede ver el código en la función principal.

```
if (Obstaculos->Count > 0) {  
    EliminarObstaculos();//Elimina obstaculos no validos  
    prepararObstaculos();//Geometria del obstaculo  
    RelacionarObstaculos();//Tracking  
    copiarObstaculos();//Copiar el vector de obstaculos obtenido  
    //en la capa de control y comprueba colisiones  
}
```

Figura 61: Código en la función principal del filtrado de obstáculos

Filtrado

En caso de que se haya encontrado algún obstáculo, se ha de realizar un filtrado de los obstáculos no validos de la siguiente manera.

```
void DataAnalysis::EliminarObstaculos()  
{  
    int ObstPequeños = 0;  
    int ObstEliminados = 0;  
    for (int p = 0; p < Obstaculos->Count; p++)  
    {  
        if (!Obstaculos[p]->Valido) {  
            Obstaculos->RemoveAt(p);  
            p--;  
            ObstEliminados++;  
        }  
        else if (Obstaculos[p]->components->Count < TAMAÑO_MINIMO_OBSTACULO)  
        {  
            Obstaculos->RemoveAt(p);  
            p--;  
            ObstPequeños++;  
        }  
    }  
}
```

Figura 62: Código fuente de la función eliminar obstáculos

Como se puede ver en la imagen anterior, el código fuente de la función EliminarObstaculos (), se han implementado dos filtros para los obstáculos no válidos.

En primer lugar los obstáculos marcados como no validos son eliminados. Un obstáculo puede ser marcado como no valido durante la segmentación, esto ocurre cuando dos obstáculos se unen, para no modificar los identificadores de obstáculo que podría llegar a suponer una gran carga de trabajo, se marca el obstáculo que es absorbido como no valido. De esta manera se elimina la posible carga de trabajo innecesaria.

El segundo requisito es que el obstáculo incluya un mínimo de puntos. Actualmente el mínimo de puntos se encuentra configurado a 20.

Preparación

Una vez realizado el filtrado de los obstáculos no válidos, se preparan los obstáculos válidos. Esto significa, que se ejecutan una serie de funciones que calculan:

- El centro del obstáculo
- La orientación del obstáculo
- La bounding box del obstáculo

Tracking

Por último, con los parámetros de cada obstáculo ya calculados se realiza el tracking de los obstáculos con los del frame anterior.

Para ello se ha extendido el método utilizado en la fase 1 del proyecto. En la cual, como se ha explicado, hay dos modos de relacionar dos obstáculos, por posición o por velocidad, como se puede ver en la imagen siguiente.

```
for (int i = 0; i < Obstaculos->Count; i++)
{
    indice = -1;
    for (int j = 0; j < ObstaculosvAnt->Count; j++)
    {
        if (ObstaculosvAnt[j]->getVelocity() >= 1)
        {
            //0.3 es la distancia maxima que puede recorrer entre barridos
            if (Obstaculos[i]->getCenter()->distanceToPoint(ObstaculosvAnt[j]->getPrediceCenter()) < (VCOCHE / 3.6) * TIEMPO_MARGEN)
            {
                relacionarVel(i, j);
                indice = j;
            }
        }
        else if (Obstaculos[i]->getCenter()->distanceToPoint(ObstaculosvAnt[j]->getCenter()) < DISTANCIA_MAXIMA )
        {
            relacionarPos(i, j, frecuencia);
            indice = j;
        }
    }
}
```

Figura 63: Código fuente del tracking de obstáculos

De la misma manera que en la fase anterior del proyecto:

1. En primer lugar, se comprueba si se ha realizado un tracking previo al obstáculo, comprobando si tiene asignada una velocidad.
2. En caso de que tenga velocidad, se comprueba si la predicción de la posición del centro del obstáculo coincide, dentro de un margen, con la posición de algún obstáculo actual. En cuyo caso, se relacionan los obstáculos por velocidad.
3. En caso de que no tenga velocidad asignada, se comprueba si la distancia de los centros está dentro de una distancia máxima predeterminada. En cuyo caso se relacionan por posición.

Relacionar por velocidad

Si dos obstáculos se relacionan por velocidad se realizan los siguientes cálculos:

```

void DataAnalysis::relacionarVel(int i, int j)
{
    Obstaculos[i]->setDirection(ObstaculosvAnt[j]->getCenter());
    Obstaculos[i]->calculatePrediceCenter();
    Obstaculos[i]->setVelocity(VCOCHE, Frecuency);
    Obstaculos[i]->calculateTimeToCollision(VCOCHE);
}

```

Figura 64: Relacionar por velocidad

Como se puede ver en su código fuente.

En primer lugar se calcula la dirección de movimiento del vehículo con los centros del obstáculo anterior y el actual.

Se calcula la predicción de la posición del dentro en el siguiente frame, mediante la interpolación del vector de dirección.

Se calcula de la velocidad de obstáculo, para lo que se necesita la velocidad actual del vehículo, para compensar su movimiento.

Y por último, se calcula el tiempo de colisión con el vehículo, mediante el corte de sus trayectorias, y el cálculo de una región de impacto y los tiempos de entrada y salida tanto del obstáculo como del vehículo de la misma.

Relacionar por posición

Para relacionar por posición se sigue el mismo método que en el caso de relacionar por velocidad, con el único cambio de que no se calcula el tiempo de colisión, debido a la baja fiabilidad del tracking por posición.

Copiar obstáculos

En este momento, todos los obstáculos han sido detectados y se ha realizado el tracking.

Una vez realizado todo el proceso se copia el vector de obstáculos calculado a la capa de control y memoria compartida con el objetivo de que el resto de componentes puedan acceder a él.

```

void DataAnalysis::copiarObstaculos()
{
    Informar("Copiar Obstaculos");
    //Controller de collision
    if (Flags[FLAG_TRATAMIENTO]) {
        Flags[FLAG_WARNING] = true;
        Informar("Error de Colision");
    }
    Flags[FLAG_TRATAMIENTO] = true;

    ObstaculosvAnt->Clear();
    ObstaculosvAnt->AddRange(Obstaculos);

    if (Flags[FLAG_OPENGL]) {
        Informar("Obstaculos->OpenGL");
        Dibujador->modificarObstaculos(ObstaculosvAnt);
    }
    Obstaculos->Clear();
}

```

Figura 65: Código fuente de la copia de obstáculos a la capa de control

En la imagen anterior se puede ver el proceso de copia de obstáculos a la capa de control. En primer lugar se realiza un control de colisiones en el cual se realiza la comprobación del flag de tratamiento para comprobar si el modulo del lector ha adelantado al análisis, lo que significa que no se cumple el requisito de tiempo real.

En caso de que se produzca una colisión se activa el flag de warning que bloqueara el sistema.

Tras ello, se pone el flag de tratamiento a true para indicar que el análisis ha terminado.

Se limpia el array de los obstáculos de la vuelta anterior y se copia el array de obstáculos de la vuelta actual en el para el análisis del siguiente frame.

Por último, se comprueba si el visualizador esta encendido, en cuyo caso se le indica que ha de redibujar los obstáculos en la interfaz de visualización. Y se limpia el array de obstáculos actual para dejarlo preparado para el siguiente análisis.

Volviendo al último paso del análisis, en caso de que se realice una pausa se limpia el array de obstáculos de la vuelta anterior ya que cuando se reactive el sistema este no será válido y podría crear problemas en el análisis.

Una vez limpiado el array se envía el thread de análisis a la función Esperar () donde se mantiene en un bucle a la espera de la continuación de la ejecución, la cual se activa a través de la interfaz gráfica.

Representación en OpenGL

En este apartado se va a explicar la configuración e implementación realizada para el modulo del visualizador.

Constructor y parámetros

Tal y como ocurre con el resto de módulos, el módulo del visualizador se ejecuta en un thread propio, el cual se incluye en la lista de threads en la capa de control a su arranque.

```
void OpenGL::threadconstructor()
{
    Informar("THREAD OPENGL");
    iniciarPuntos();
    iniciarObstaculos();
    int argc = 0;
    char **argv;
    glutInit(&argc, argv);
    trabajo();
    // Start GLUT event processing loop
    glutDisplayFunc(display);
    glutMainLoop();
}
```

Figura 66: Constructor del visualizador

En la imagen anterior se puede ver el código fuente del constructor del visualizador, el cual ha sido desarrollado en OpenGL.

Para comenzar se han de inicializar las listas de puntos y obstáculos, los cuales, debido al uso de OpenGL, deben ser objetos estáticos. Las funciones encargadas de inicializar las listas únicamente inicializan las listas con puntos y obstáculos en la posición (0, 0, 0).

Por otro lado glutInit, glutDisplayFunc y glutMainLoop son funciones internas de OpenGL que son necesarias para el inicio de la visualización.

Por último, explicar las funciones trabajo y display.

La función trabajo contiene una serie de funciones de OpenGL necesarias para la inicialización de la ventana. Como se puede ver a continuación.

```
void trabajo()
{
    centerOnScreen();
    glutInitWindowSize(window_width, window_height);
    glutInitWindowPosition(window_x, window_y);
    glutInitDisplayMode(GLUT_RGBA | GLUT_DOUBLE);
    glutCreateWindow(window_title);
    init();
    glutKeyboardFunc(keyboard);
    glutSpecialFunc(specialKeys);
    glutTimerFunc(2000, timerFunc, 0);
}
```

Figura 67: Función trabajo del visualizador

Mediante esta serie de funciones se consigue lo siguiente:

- Centra la ventana de visualización en el centro de la pantalla
- Define el tamaño de ventana en 750 x 750
- Guarda la posición de la ventana en dos variables para que su posición pueda ser modificada
- Define el modo de color de la ventana en RGB e indica que debe utilizar doble buffer para dibujar
- Crea la ventana
- Inicializa el color de fondo de la ventana
- Activa las pulsaciones de teclado y teclas especiales para controlar la visualización de la ventana, las cuales se explicaran más adelante
- Por último, se define el tamaño del bucle de tiempo para el dibujado, ya que este no se puede realizar de manera totalmente continua ya que bloquearía el sistema.

La función display es la encargada de dibujar las estructuras básicas en la ventana para que quede de la siguiente manera.

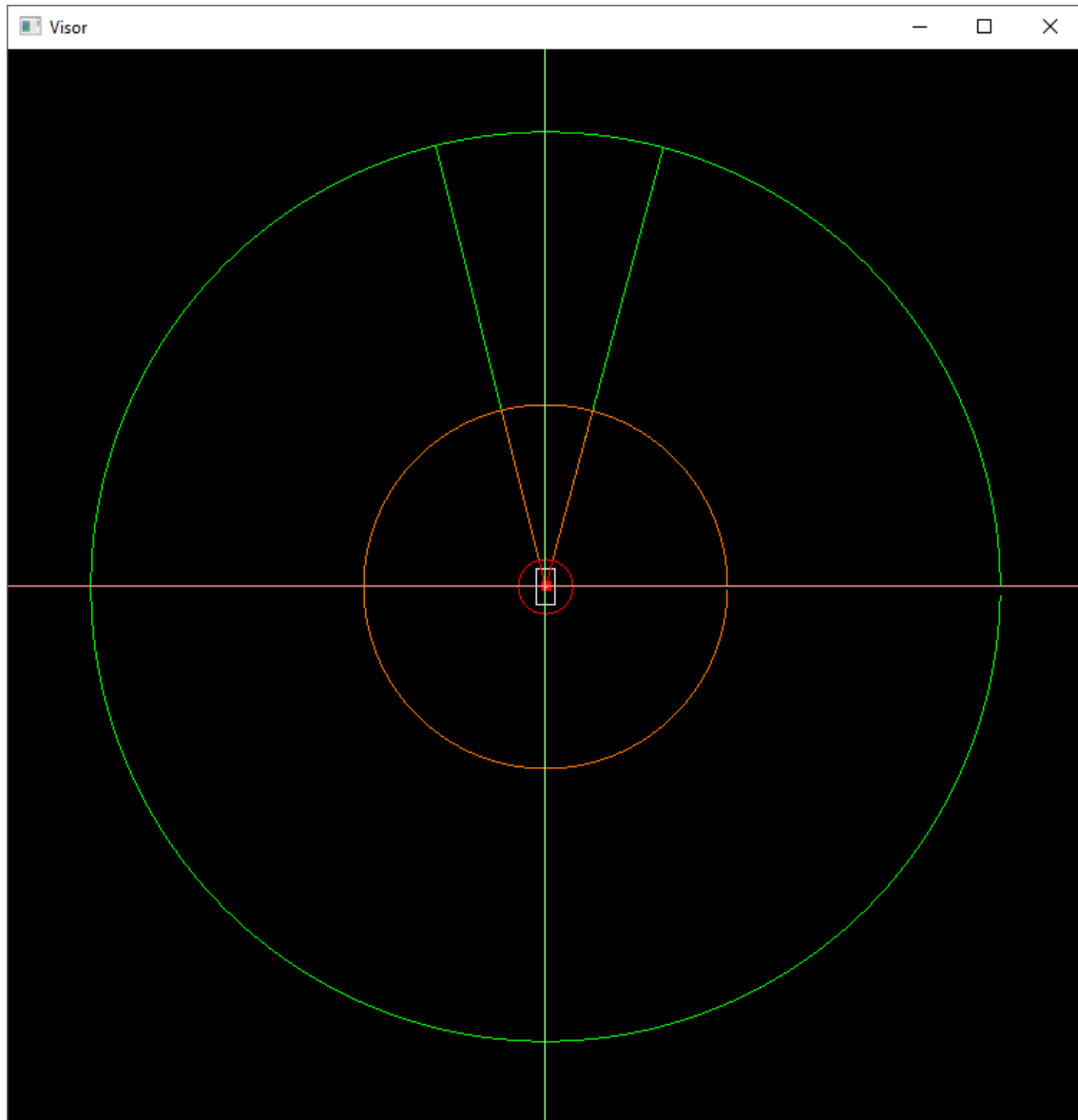


Figura 68: Estructura básica de la ventana del visualizador

Esta función dibuja:

- Los ejes X, Y, Z
- El vehículo
- Los 3 radios de alerta a 3, 20 y 50 metros
- Dibuja el cono de acción en la parte frontal del vehículo
- Dibuja todos los puntos y obstáculos posibles en el punto (0, 0, 0)

Una vez dibujada la estructura principal, se han definido dos funciones dibujar puntos y dibujar obstáculos, mediante las cuales los módulos del lector y análisis indican al visualizador que un nuevo frame está listo o que los obstáculos ya han sido calculados.

Modificar puntos

```
void OpenGL::modificarPuntos(List<Punto3D>^ listEntradaPuntos)
{
    Informar("MODIFICAR PUNTOS" + listEntradaPuntos->Count);
    listo = false;
    if (listEntradaPuntos->Count < puntos->Count) {
        limpiarListas(listEntradaPuntos->Count);
    }
    for (int recorridoP = 0; recorridoP < listEntradaPuntos->Count; recorridoP++) {
        puntos[recorridoP]->setCoordinatesX(listEntradaPuntos[recorridoP]->getCoordinatesX());
        puntos[recorridoP]->setCoordinatesY(listEntradaPuntos[recorridoP]->getCoordinatesY());
        puntos[recorridoP]->setCoordinatesZ(listEntradaPuntos[recorridoP]->getCoordinatesZ());
    }
    puntosAnterior = listEntradaPuntos->Count;
}
```

Figura 69: Código fuente de la función modificar puntos

Como se puede ver en el código fuente, en la imagen anterior, la función modificar puntos recibe una lista de puntos. En primer lugar limpia la lista de puntos actuales y la de obstáculos, lo que significa que los mueve de nuevo al punto (0, 0, 0). Tras ello, recorre la lista de puntos y mueve los puntos de su lista local a las posiciones de la lista de entrada.

Modificar obstáculos

La función de modificar obstáculos funciona de manera similar a la de modificar puntos. La función recibe una lista de obstáculos y comienza limpiando la lista de obstáculos actuales. Tras ello, utiliza los valores de los vértices de la bounding box para mover los obstáculos de su lista local que están colapsados en el (0, 0, 0), a sus posiciones en el espacio.

Teclas especiales

Se han definido las siguientes teclas de control para el control de la interfaz de visualización:

- H: imprime las instrucciones de uso de las teclas por la interfaz gráfica.
- Teclas '5', '6' y flechas se utilizan para la rotación de la cámara.
- Teclas '7', '8', '-', '+', '=' se utilizan para quitar o poner zoom.
- Tecla ESC cierra el visualizador
- Tecla SPACE vuelve a la posición inicial

El uso de estas pulsaciones de teclado o teclas especiales se han definido en funciones propias de OpenGL llamadas keyboard () y specialKeys ().

Interfaz de pruebas Off-line

Por último, en esta fase del proyecto se ha desarrollado una interfaz de pruebas offline que trabaja con los logs generados desde la interfaz gráfica normal.

Mediante esta interfaz se puede simular una conducción real tanto en velocidad del vehículo como la nube de puntos. Se ha desarrollado con el objetivo de reducir costes, y para facilitar la tarea de desarrollo.



Figura 70: Interfaz offline

En la imagen anterior se muestra la interfaz offline, a continuación se explicara el protocolo para la utilización de la interfaz y los modos de uso.

En primer lugar, se ha de seleccionar el archivo de log que debe tener la estructura definida por el generador de log de la interfaz gráfica.

Tras ello, se puede activar el sistema mediante el botón “activar” con lo que iniciara la lectura del archivo y el análisis.

Se puede activar el botón de pausa en cualquier momento de la ejecución y reactivar la ejecución de la misma manera.

Puesto que no hay comunicación con el vehículo la velocidad del mismo ha de ser indicada manualmente. Tras indicarla se ha de pulsar el botón Set velocidad para que el cambio surta efecto.

Por último, la interfaz cuenta con un botón “lock”. Mediante el cual se le puede indicar el sistema que se pare en el frame indicado a su derecha.

El botón analizar, que no ha sido explicado, sirve para ejecutar un análisis de la nube de puntos en caso de utilizar el botón “lock”.

Fase 3: Desarrollo de un sistema de fusión de datos

En esta fase se ha desarrollado una implementación de un sistema de fusión de datos, este realiza una fusión inteligente entre objetos que vienen de un láser scanner y una cámara Mobileye.

El sistema de fusión está dividido en 3 fases:

- El pre proceso cuya mayor parte ha sido desarrollada por Valeo. Este consiste en realizar: reconocimiento de objetos, segmentación y tracking de los objetos.

Aparte las mediciones incluyen un timestamp que sincroniza los sensores. Mediante software.

- El Matching, se ha desarrollado un algoritmo que consiste en encontrar las posibles asociaciones entre los objetos de los sensores y seleccionar la mejor asociación posible entre ellos.
- La estimación, se ha implementado un algoritmo de estimación Bayesiana cuyo resultado es la fusión de los datos de dos objetos que han sido relacionados en la fase anterior.

Problemas encontrados

Los problemas que han sido encontrados durante el desarrollo de este proyecto son los siguientes:

Sincronización

Mientras se desarrollaba el algoritmo de fusión se ha advertido que existe un problema de sincronización en los objetos.

Este problema es que las entradas de los datos del láser y la cámara no ocurren de manera simultánea, en términos de sincronización. En vez de eso las entradas utilizan una marca de tiempo, timestamp, usando un reloj externo a los sensores, un reloj global. Esto significa que los datos de los sensores llegan al módulo de análisis de manera asíncrona.

Esta ausencia de sincronización de hardware supone un problema para el sistema de fusión en cuanto a que la fusión podría no relacionar datos ocurridos en el mismo momento.

En este caso, esto significa que las entradas de los dos sensores que vienen uno de la cámara y otro del láser no representan el mismo escenario.

El impacto en el proceso de fusión varía dependiendo del salto de tiempo que haya entre las dos lecturas.

Asociaciones

El problema de la asociación de datos es el problema principal en la fusión.

El núcleo de este problema se encuentra en determinar que objeto del láser coincide con un objeto de la cámara.

Este es el paso más crítico porque si la asociación no se realiza correctamente, esto causara un fallo en la estimación y en el proceso de tracking.

En este proceso se han de considerar todas las posibles ambigüedades en la asociación de objetos.

Ambigüedades

En primer lugar, se asume que el proceso de reconocimiento de obstáculos realizado por Valeo se ha realizado correctamente, lo que significa que es imposible que un

objeto real haya sido reconocido por su algoritmo como 2 o más objetos, un objeto real no se ha dividido en 2 o más objetos detectados.

Esto no significa que se asume que ha habido un 100% de ratio de detección, que todos los objetos hayan sido reconocidos, ni que los objetos hayan sido perfectamente definidos (tamaño, velocidad...). Esto solo significa que cuando un objeto es detectado, este objeto puede tener algún error en sus mediciones pero este error es suficientemente pequeño como para que el objeto sea reconocido.

Con estas limitaciones hay algunas ambigüedades que hay que manejar:

- Varios objetos del láser podrían ser asociados con el mismo objeto de la cámara.
- Varios objetos de la cámara podrían ser asociados con el mismo objeto del láser.
- Cualquier objeto tanto del láser como de la cámara podría no ser asociado con ningún objeto del otro grupo.

Si no se consideraran las anteriores restricciones habría que desarrollar un filtro que considere que un objeto real podría haber sido detectado como varios objetos separados.

Estimación

Cuando se ha definido de manera segura que dos objetos, uno del láser y uno de la cámara, son el mismo objeto hay que fusionar la información para obtener una información más precisa y creíble del objeto real.

El problema de la estimación consiste en calcular los nuevos parámetros del objeto que es resultado de la asociación de un objeto proveniente del láser y uno proveniente de la cámara.

Soluciones técnicas

A continuación se van a exponer las soluciones desarrolladas e implementadas para los problemas expuestos anteriormente.

Adaptador del frame rate (Pre proceso)

Con el objetivo de resolver el problema de sincronización, se ha implementado el siguiente método basado en el desarrollo de un componente de sincronización.

Este componente es el encargado de generar un marco de tiempo común para los dos sensores mediante la alineación de tiempos y datos.

Para realizar esto, se utiliza un proceso simple de interpolación. De hecho a cada objeto, proveniente de cualquiera de los dos sensores, se le ha realizado previamente un tracking, por lo que dispone de un vector velocidad, el cual da la información de dirección y velocidad.

Se asume que dicho vector velocidad del objeto es constante, dentro del periodo del tiempo entre un frame y el siguiente del sensor, por lo que se puede interpolar la

posición del objeto en cualquier momento. Concretamente, se ha comprobado que el máximo tiempo de diferencia entre la llegada de los datos de un sensor y del otro es de 300 ms, por lo que se puede asumir que en este periodo de tiempo la velocidad y dirección son constantes.

En consecuencia, la posición de un objeto en movimiento con velocidad 's' y tiempo 't' puede ser definida mediante la siguiente formula:

Ecuación 6: Ecuación de posición respecto a la velocidad y tiempo

$$p = p_0 + s * t$$

Donde:

p_0 = posición actual del objeto

S = velocidad

T = espacio de tiempo entre la recepción de los datos de un sensor y del otro

P = posición del objeto en el marco de tiempo común

El resto de parámetros del objeto no cambian en este proceso.

Gracias a este componente se puede obtener un data set de objetos que pertenece al mismo escenario temporal y por lo tanto mejora el proceso de fusión.

Además, mediante este componente es posible adaptar el frame rate a la capacidad computacional del equipo en el que el sistema este ejecutándose.

Graficas de los resultados obtenidos

Las siguiente graficas representan el funcionamiento del algoritmo de pre proceso.

En las gráficas se encuentran representados el objeto del láser con id = 71 y el objeto de la cámara con id = 11 entre los tiempos de 803000 ms y 807000 ms con lo que representan 4 segundos de captura, con el objetivo de mostrar su funcionamiento con ambos sensores.

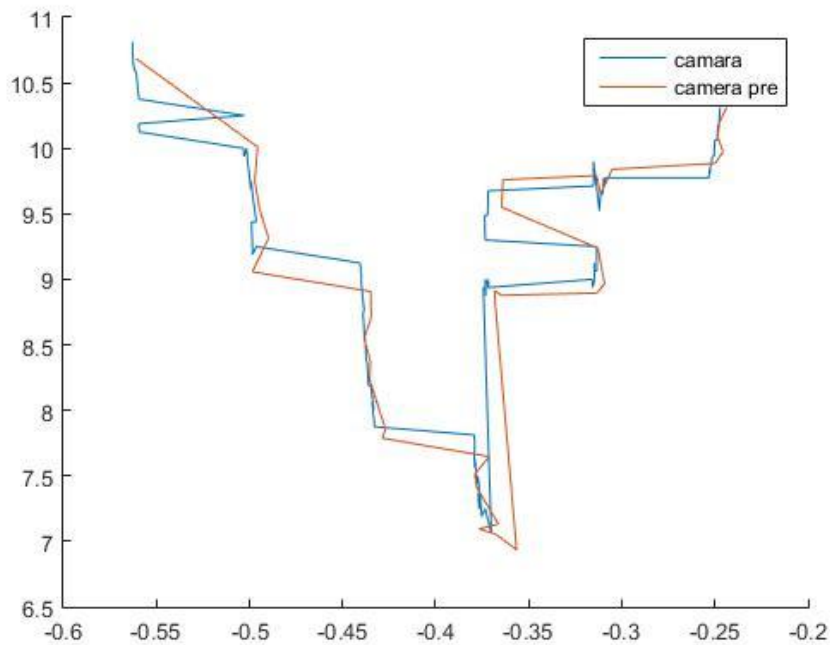


Figura 71: Funcionamiento del pre proceso en un objeto de la cámara

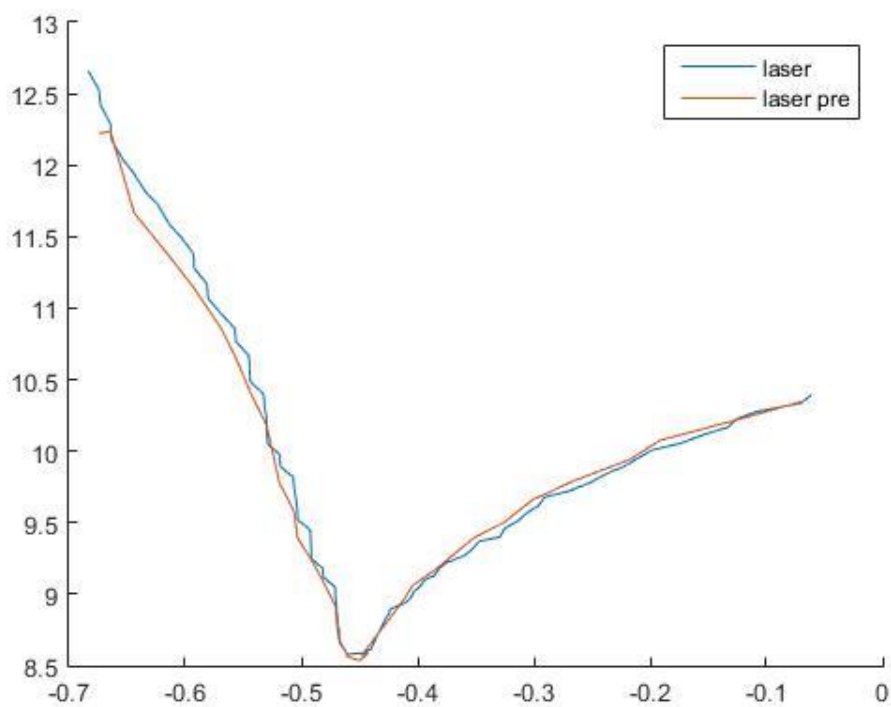


Figura 72: Funcionamiento del pre proceso en un objeto del laser

En las gráficas anteriores se puede ver el movimiento del objeto original, la línea azul, y el movimiento después de realizar el pre proceso, la línea naranja.

Se puede observar que el componente resulta útil ya que hace que el movimiento de los obstáculos sea más suave, y reduce posibles errores en la medida.

Este no es el método más eficiente pero es una buena aproximación teniendo en cuenta el tiempo consumido por este proceso.

A continuación se muestra la comparación de las gráficas anteriores.

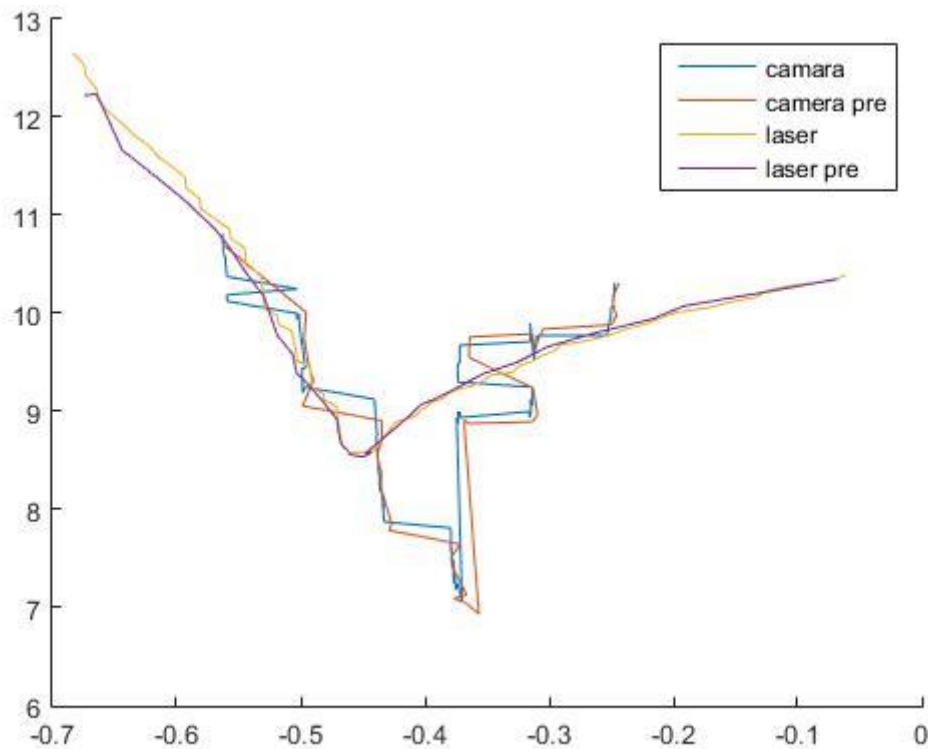


Figura 73: Comparación de graficas del pre proceso

Lo representado en la gráfica anterior es la superposición de las dos graficas previas.

Esta grafica es importante por la siguiente razón. Ambas detecciones, la del láser y la de la cámara, corresponden al mismo objeto real, lo cual ha sido comprobado manualmente.

Sabiendo esto se puede observar que existe una diferencia importante entre las mediciones de la cámara y el láser, esto se debe a que la cámara o el reconocimiento de sus objetos, tienen un error importante, concretamente alrededor de 1,5 metros de error. Esto condicionara la estimación final.

A pesar de ello el funcionamiento del algoritmo no se ve afectado. Lo que demuestra que el sistema es capaz de manejar los fallos en el data set de entrada.

Asociación, Matching

Con el objetivo de resolver el problema de la asociación, se ha desarrollado una solución en dos pasos.

Matching_1

El trabajo del primer paso es, para cada objeto del láser, buscar todas las posibles asociaciones, sin importar el tamaño. A continuación se describe el método seguido para este paso.

En primer lugar, para todos los objetos, tanto del láser como de la cámara, se calculan sus respectivas bounding boxes. De hecho cada parámetro del objeto puede ser considerado como una distribución Gaussiana, cada parámetro está compuesto por una medida y una varianza o sigma. Por lo tanto se puede considerar que la medida es la media de la distribución y su sigma es la varianza.

Con ello se puede calcular un área en la cual es muy probable que se encuentre realmente el objeto.

Esto se implementa de la siguiente manera:

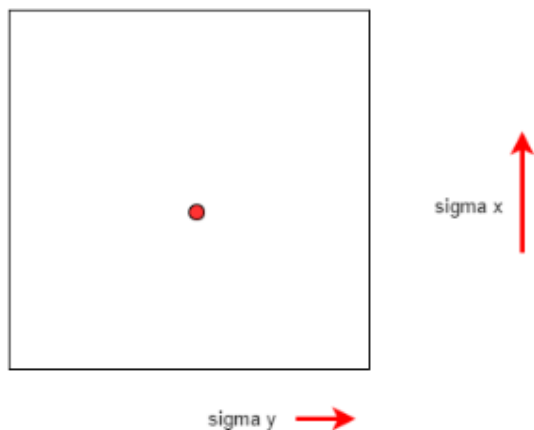


Figura 74: Calculo de la bounding box extendida 1

En primer lugar se coge el parámetro y sus varianzas. En este caso para el parámetro de la posición del objeto se toman las coordenadas de su centro y la sigma de la coordenada x e y.

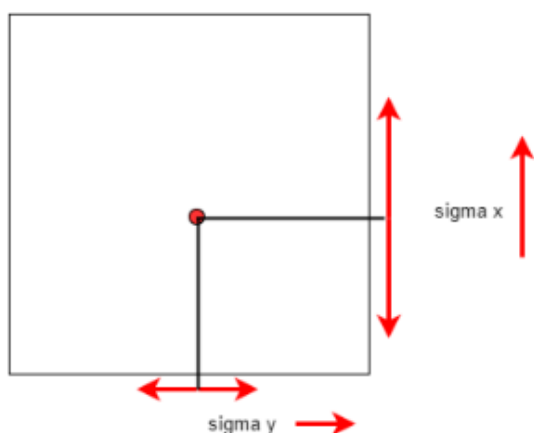


Figura 75: Calculo de la bounding box extendida 2

Ya que la varianza es la medida de dispersión de una variable, en este caso cada una de las coordenadas de la posición, se puede extender el parámetro mediante las sigmas.

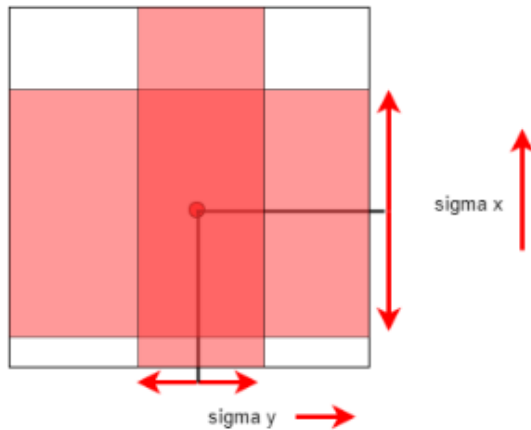


Figura 76: Calculo de la extended bounding box 3

Ya que se consideran que las coordenadas de la posición son variables independientes, se obtiene un resultado similar a la imagen anterior. En la cual tenemos un área de x e y dada por la extensión del parámetro con sus varianzas.

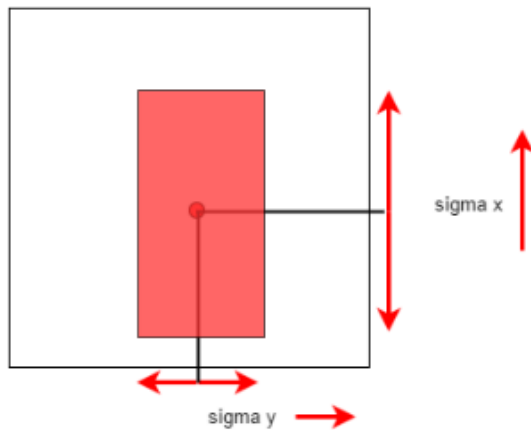


Figura 77: Calculo de la extended bounding box 4

Por último, de la intersección de las dos áreas generadas se obtiene el área que se buscaba. En la cual hay gran posibilidad de que se encuentre el objeto, teniendo en cuenta el posible error del sensor.

Una vez que todas las bounding box extendidas han sido calculadas, se puede comparar cada objeto del láser con cada uno de los objetos de la cámara mediante ellas.

Si existe intersección entre sus respectivas bounding box extendidas, se puede guardar este par de objetos como una posible asociación, con el objetivo de ser procesada en el siguiente paso.

El resultado de este paso es una matriz con n (número de objetos del láser) filas y m (número de posibles asociaciones con este objeto) columnas.

		Camera ids							
Laser objects		1	2	3	-1	-1	-1	-1	-1
		1	2	5	8	-1	-1	-1	-1
		3	4	-1	-1	-1	-1	-1	-1
		-1	-1	-1	-1	-1	-1	-1	-1

Figura 78: Resultado del primer paso de la asociación

Matching_2

En este segundo paso se calcula la similitud entre dos objetos. Aquí se compara cada objeto del láser con los objetos de la cámara en su lista de posibles asociaciones, proporcionada por el paso 1.

Algoritmo MHT

Inicialmente se consideró la implementación del algoritmo MHT con el objetivo de desarrollar esta parte del sistema de fusión.

El algoritmo MHT consiste en generar un árbol de posibles decisiones de asociación. A cada asociación se le asigna una puntuación que se calcula mediante la distancia estadística de Mahalanobis entre dos objetos, uno de cada sensor.

Tras estudiar el funcionamiento del algoritmo y sus posibles implementaciones, fue descartado ya que este solo considera la posición del obstáculo como medida de correspondencia.

Se consideró que no era suficiente ya que los objetos contaban con gran cantidad de información útil. Esta información no utilizada por este algoritmo puede ser utilizada para realizar la asociación incluso a pesar de que requiere más memoria.

Lamentablemente, Rtm maps no puede manejar este coste de memoria por lo que el componente se bloquea.

Con el objetivo de implementar este algoritmo es necesario la utilización de estructuras de memoria dinámicas que Rtm maps no puede manejar entre componentes.

Por ello se ha optado por desarrollar un algoritmo propio.

Algoritmo de puntuación

Para medir la puntuación de una asociación, se puede comparar cada parámetro útil del objeto como, tipo de objeto, posición, velocidad, solapamiento, tamaño de manera separada, dado una puntuación a cada parámetro. Esta puntuación viene dada por la coincidencia de este parámetro entre dos objetos.

Tras ello, utilizando una media ponderada se puede calcular la puntuación global de la asociación.

Comparación de similitud entre dos parámetros

El primer paso para comparar dos parámetros es crear sus bounding boxes, tal y como se ha visto anteriormente.

Con las bounding boxes del parámetro, uno del láser y uno de la cámara se puede comparar el área de las bounding boxes y medir la coincidencia con la siguiente fórmula:

$$Puntuacion = \frac{\cap}{\cup}$$

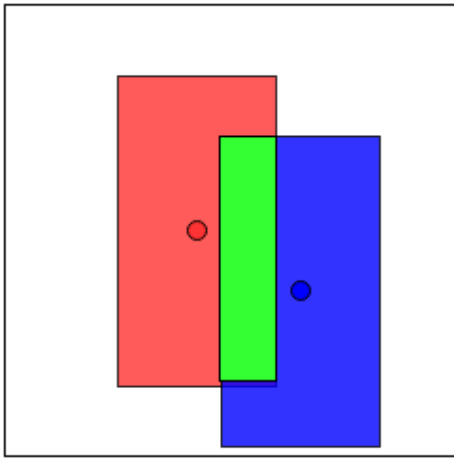


Figura 79: Cálculo de la puntuación de coincidencia de un parámetro

En la imagen anterior se puede ver como se utiliza este método para comparar la coincidencia entre las posiciones.

El área verde correspondería a la intersección.

Y la suma de las áreas de los parámetros menos su intersección correspondería a la unión.

De hecho la fórmula corresponde a la intersección del área de las distribuciones Gaussianas de estas variables, con lo que realmente se está realizando un cálculo de la distancia de Mahalanobis entre las variables.

Si las variables fueran dependientes el cálculo de la puntuación no podría utilizar esta fórmula, ya que en ese caso el área de la distribución Gaussiana que debe ser comparada sería una elipse y no un rectángulo. Pero, en este caso las variables se consideran independientes. Además aun si fueran dependientes habitualmente se usa esta fórmula con el objetivo de reducir la complejidad de los cálculos.

Con este método se puede calcular la puntuación de coincidencia de todos los parámetros que tiene el objeto.

La puntuación de cada asociación es calculada como una media ponderada de las puntuaciones de cada parámetro del objeto. Los pesos de la media son actualmente determinados de manera empírica, manual.

En cualquier caso, se han definido unas restricciones para seleccionar la asociación:

- La puntuación de la asociación ha de ser mayor a 20 (%)
- Solo puede haber una asociación para cada objeto.

Por lo tanto, se han de resolver las ambigüedades creadas por este proceso.

En este momento, al final del cálculo de puntuaciones se tiene una matriz con la siguiente forma.

Laser Objects	190	480	240	-10
	350	125	-10	-10
	495	-10	-10	-10

Score of the association

Camera ID

Figura 80: Matriz de posibles asociaciones y su puntuación

Ambigüedades

Tras este proceso, todos los obstáculos del láser están asociados con su mejor posible asociación. Con el proceso podría pasar que varios obstáculos del láser hayan sido asociados con el mismo obstáculo de la cámara. Este es un problema que hay que abordar.

Para resolver esta situación, se busca cuál de ellos tiene la mejor puntuación y a todos los demás se les intenta asociar con su siguiente mejor asociación posible.

Durante este proceso de seleccionar la mejor asociación posible podría pasar que una asociación mejor para algún obstáculo haya sido liberada.

Por ello se ha de hacer una revisión de las posibles asociaciones de cada obstáculo, para buscar una mejor posible asociación, que en este momento estuviera liberada.

A continuación se muestra el código para realizar este proceso:

```

do
{
    while (findAmbiguities())
    {
        for (int i = 0; i < joined.size(); i++)
        {
            posAmb = findAmbiguities(i);
            if (posAmb != -1)
            {
                if (joined.vector[i][2] >= joined.vector[posAmb][2])
                {
                    selectNextAssociation(posAmb);
                }
                else
                {
                    selectNextAssociation(i);
                }
            }
        }
    }
} while (lastCheck());

```

Figura 81: Código de búsqueda de ambigüedades

En la imagen anterior se pueden observar todos los bucles necesarios para realizar la mejor asociación posible.

El objetivo de utilizar este algoritmo propio es que se busca encontrar coincidencias entre dos obstáculos teniendo en cuenta todos los parámetros posibles y no solo las posiciones de los obstáculos como se implementa en los algoritmos MHT clásicos. Además, este algoritmo es fácilmente escalable, ya que puede utilizar nuevos parámetros que podrían calcularse si cambiara el algoritmo previo de reconocimiento de obstáculos.

Estimación

Esta sección está dedicada a la explicación del cálculo de la fusión, estimación de los obstáculos que han sido asociados en el paso previo.

El objetivo de este módulo es mejorar los parámetros de los objetos asociados gracias a la información redundante y/o complementara dada por el láser y la cámara. Para realizar el proceso de fusión se ha optado por implementar una técnica de fusión Bayesiana.

Teoría Bayesiana

Como se ha explicado antes se puede entender cada parámetro como una distribución Gaussiana, esto significa que se tiene la media y la varianza de este parámetro. Con esto se puede calcular el parámetro resultante aplicando inferencia Bayesiana.

Mediante estas fórmulas:

Ecuación 7: Calculo de la media mediante inferencia Bayesiana

$$\hat{\phi}(x_1, x_2) = \frac{\sigma_2^2}{\sigma_1^2 + \sigma_2^2}x_1 + \frac{\sigma_1^2}{\sigma_1^2 + \sigma_2^2}x_2.$$

Ecuación 8: Calculo de la varianza mediante inferencia Bayesiana

$$\text{var} [\hat{\phi}(X_1, X_2)] = \frac{\sigma_1^2 \sigma_2^2}{\sigma_1^2 + \sigma_2^2}$$

Asumiendo que las variables X_1 y X_2 son variables independientes y que la varianza del error es σ_i^2 .

Con esta formulas se obtiene una nueva media y una nueva varianza para un parámetro x resultado de la fusión de los parámetros X_1 y X_2 . Este resultado está directamente relacionado con la precisión de los sensores así como de la precisión del algoritmo de reconocimiento de objetos.

Utilizando el mismo ejemplo presentado en la sección del pre proceso se obtiene el siguiente resultado.

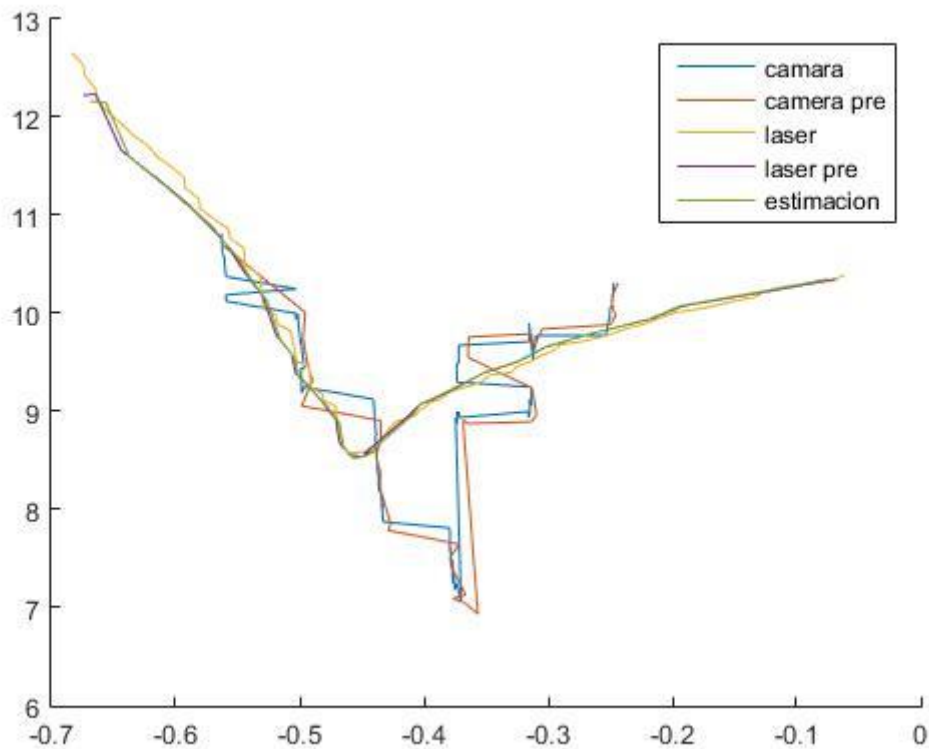


Figura 82: Resultado de la fusión

Se puede observar que el resultado de la estimación es prácticamente el mismo que la salida del láser. Esto ocurre debido a que la varianza de la cámara es mucho mayor que la varianza del láser.

Si las varianzas fueran parecidas el resultado sería una combinación entre la información proporcionada por el láser y la cámara.

Debido al error en el parámetro de la varianza para los obstáculos de la cámara, se ha de modificar la anterior formula de la estimación para el cálculo de la varianza por la siguiente, la cual ha sido deducida de las especificaciones técnicas de la cámara Mobileye.

Ecuación 9: Fórmula de cálculo de la varianza modificada

$$\hat{\phi}_P(x_1, x_2) = \frac{\sigma_2^2}{\sigma_1^2 f_1^2(x_1) + \sigma_2^2} x_1 + \frac{\sigma_1^2 f_1^2(x_1)}{\sigma_1^2 f_1^2(x_1) + \sigma_2^2} x_2.$$

Ecuación 10: Aclaración en la fórmula del cálculo de la varianza modificada

$$f_1(x) = \max(0.1x, 2)$$

Esto ha sido propuesto con el objetivo de mejorar la precisión de la estimación mientras las varianzas sean erróneas.

Resultados

Ya que no se cuenta con un ground truth con la base de datos con la que se ha trabajado no se puede medir cuantitativamente la precisión del algoritmo. En cualquier caso, los resultados del algoritmo son satisfactorios en comparación al resultado de otros algoritmos de fusión.

Por otro lado, se podría mejorar la implementación del algoritmo mediante el ajuste de los pesos de los parámetros el paso Matching_2.

Aclaración de los módulos desarrollados

Para este proyecto se ha desarrollado un módulo de Rtm maps para cada fase del algoritmo con el objetivo de poder hacer un seguimiento más exhaustivo del funcionamiento del algoritmo en fase de desarrollo.

El algoritmo final ha sido implementado en un solo modulo con el objetivo de facilitar su utilización en proyectos más grandes además de por términos de eficiencia, ya que se elimina el paso de mensajes entre módulos de Rtm maps lo que reduce el tiempo de ejecución del algoritmo y facilita el manejo de memoria.

Conclusiones

En este apartado se expondrán las conclusiones a las que se ha llegado con la realización de este proyecto.

En primer lugar, con este proyecto se ha mostrado una pequeña parte de lo que se puede llegar a hacer, con sensores tanto Lidar como cámaras. Se ha dado a conocer la relativa sencillez para realizar análisis en los datos proporcionados por un sensor Lidar, así como su funcionamiento, precisión y rango de trabajo.

A pesar de haber trabajado en este proyecto con sensores de alta precisión, y por lo tanto muy caros. Se ha intentado mostrar una de las técnicas más utilizadas en el campo de la robótica, la estimación Bayesiana. Tanto con una sencilla ecuación de estimación, como con sus versiones más desarrolladas como un Filtro de Kalman. Mediante las cuales se pueden obtener resultados realmente precisos a partir de fuentes no tan precisas.

En el ámbito personal, este proyecto ha servido para, en algunos casos afianzar conocimientos adquiridos en la carrera, y en otros para aprender nuevos conocimientos los cuales no podrían haber sido abordados sin la base de aprendizaje adquirida durante la carrera.

Gracias a este proyecto, que supone el inicio de la vida profesional, se ha adquirido una nueva visión del funcionamiento de un proyecto real. La cual supone una visión más profesional, más allá de la visión académica adquirida durante la carrera. Donde realmente se ha mostrado la complejidad de organización de un proyecto completo, su planificación y la manera de llevarlo a cabo. Durante la realización de este proyecto se han adquirido nuevas habilidades que solo se adquieren con práctica y trabajo.

En conclusión, gracias a la posibilidad de trabajar en un proyecto real enfocado de una manera profesional ha resultado una gran experiencia, que supone la finalización de mi formación dentro del ámbito universitario y el inicio de mi formación dentro del ámbito profesional.

Trabajo futuro

En cuanto al trabajo futuro, cabe destacar que este trabajo nunca estará finalizado. Se ha de ir modificando y mejorando con nuevas técnicas y tecnologías.

Puesto que este proyecto ha supuesto el desarrollo de un trabajo futuro continuo, con nuevos objetivos tras la finalización de cada fase. Es lógico continuar por el mismo camino en el futuro.

El comienzo de la fase 1, empezó con el desarrollo de una detección de obstáculos con un láser scanner de 1 canal, cuya finalización arranco la fase 2. Con el desarrollo de un sistema completo de detección de obstáculos con un láser scanner de 16 canales.

Una vez finalizado esta fase, el paso lógico era incluir nuevos sensores, con lo que comenzó la fase 3 y el desarrollo de un sistema de fusión de datos.

Ahora una vez terminada esta fase el siguiente paso es continuar con el desarrollo del resto de componentes de un vehículo autónomo como la comunicación con el vehículo, adquisición de datos de nuevos sensores o desarrollo de un sistema de auto localización.

Pero en el plano de tiempo más inmediato el trabajo a realizar es el siguiente:

- Desarrollo de un sistema de configuración del láser VLP-16 a través de la interfaz del sistema en tiempo real.
- Desarrollo de un algoritmo de reconocimiento de obstáculos.
- Desarrollo de un módulo de machine learning para el ajuste de los pesos de la función de cálculo de la puntuación de la asociación en la fase 3.
- Desarrollo de un filtro de Kalman para sustituir la interpolación en el pre proceso de la fase 3.
- Desarrollo del sistema de fusión en visual estudio de manera que pueda ser unido al sistema principal

Estas 5 líneas son las tareas principales, que se deberán abordar antes del paso a desarrollar nuevos módulos del sistema. A pesar de que con el avance tan rápido de la tecnología los módulos desarrollados requerirán una actualización en un corto periodo de tiempo. Por ejemplo con el auge de la tecnología de laser de estado sólido, más rápido y preciso que el láser utilizado durante el proyecto.

Impacto social y ambiental

En un futuro, más próximo de lo que la sociedad espera, comenzaran a verse todas las tecnologías que se han visto en este proyecto y muchas más en el ámbito de los vehículos autónomos. Esto generara un gran impacto social en la medida que cambiara el modo de ver la conducción hoy en día, el medio de transporte más utilizado en la actualidad.

Con estas nuevas tecnologías que están en desarrollo, se espera reducir el número de accidentes en las carreteras considerablemente. Esto se debe a que como se puede ver en todas las campañas de concienciación que lleva a cabo la dirección general de tráfico, tanto en nuestro país como en muchos otros, la principal causa de accidente es el ser humano.

Aun así, la sociedad no está preparada para confiar ciegamente en una máquina, es por esto que los últimos intentos de probar los sistemas en entornos reales, de empresas punteras en el sector como Uber o Baidu, están viendo dificultades en conseguir los permisos necesarios de los gobiernos.

La implantación de estos sistemas en la sociedad actual va a ser lenta y progresiva. Los primeros intentos de implantación son de la empresa Uber que permite a los vehículos circular de manera autónoma pero con una supervisión constante de un conductor profesional.

Por otro lado, más allá del impacto social, el impacto medioambiental va a ser de una importancia considerable por diversas razones. Por una parte permitirá reducir el tiempo de los viajes en coche ya que el sistema no necesita parar cada 2 horas a descansar. Esto conlleva un menor gasto de combustible y su consecuente reducción en la contaminación. Por otra parte, ya que los sistemas diseñados para vehículos autónomos controlan todos los ámbitos del vehículo, conseguirán una conducción más eficiente y por tanto menos contaminante.

Por último, en el momento que se consiga una implantación total de los vehículos autónomos, y ya que se están desarrollando interfaces de comunicación entre vehículos, estos se podrán comunicar entre si y conseguir un nivel de optimización en el tráfico que no se puede concebir mediante la conducción humana. Se eliminaran problemas como atascos por diversas causas como una mala incorporación o una frenada inesperada. También serán capaces de modificar las rutas automáticamente según la densidad de tráfico ya que será posible conocerla en tiempo real. Además, los límites de velocidad no serán necesarios, mas alla que por el ahorro de combustible, o serán ampliados considerablemente, ya que actualmente estos limites están definidos con el fin de garantizar una conducción segura, en cuanto a que un humano sea capaz de reaccionar ante un posible percance, lo que no será necesario con los vehículos autónomos.

En conclusión, la introducción de los vehículos autónomos en la sociedad supondrá un gran impacto pero muy beneficioso tanto para la sociedad como para el medio

ambiente. Reduciendo el número de accidentes y optimizando la conducción, con lo que se reducirá el consumo y la contaminación.

Responsabilidad ética y profesional

En este ámbito de los vehículos autónomos, como en muchos otros, en los que se desarrollan sistemas críticos, que son aquellos sistemas cuyos errores ponen en peligro vidas humanas, se tiene un alto grado de responsabilidad.

Estos sistemas autónomos que están desarrollándose hoy en día, que están tan vigilados por la sociedad, pronto serán la norma general, y estarán instalados en todos los vehículos. Pero esto conlleva tanto ventajas como desventajas. Por un lado proporcionan confort y seguridad, pero por otro lado un pequeño error en el sistema puede volverse fatal. Es por esto por lo que la sociedad actual es tan reticente a sus pruebas en entornos reales.

En la actualidad todas las empresas y centros de investigación implicados en este desarrollo se toman muy en serio esta responsabilidad. Por ello se desarrollan baterías de pruebas realmente exhaustivas con el fin de poder asegurar que un sistema es seguro.

Es por esta razón por la que en este proyecto, como se puede observar en el apartado de trabajo realizado, se ha tenido especial control con las posibles caídas de los elementos del sistema así como la facilidad de que el resultado del análisis sea una frenada de emergencia en caso de peligro inminente, con un margen de frenada mayor de lo estrictamente necesario.

En conclusión, queda un largo camino por recorrer hasta la implantación de sistemas completamente autónomos en entornos reales, ya que para llegar a este punto se han de desarrollar aún más los sistemas para poder garantizar la seguridad, además de ser necesaria la creación de un organismo que controle y supervise esta nueva tecnología.

Referencias

- Arnanz, J. d. (2013). Análisis de Datos Obtenidos con un Láser Rotativo. Madrid.
- Bosch. (2005). Manual de la técnica del automóvil. Robert Bosch GmbH.
- García, D. T. (2011). Desarrollo e Implementación de un Sistema De Adquisición del Entorno De Conducción Mediante Láser Escáner. Madrid.
- Naranjo, J. E., Jiménez, F., & García, F. (2011). Transportation Research. Madrid: Elsevier.
- Naranjo, J. E., Jiménez, F., & García, F. (2012). An Improved Method to Calculate the Time-to-Collision of Two Vehicles. New York: Springer Science.
- Rodríguez, C. M. (2010). Identificación de Obstáculos y de su Cinemática a Partir de la Fusión sensorial en un Vehículo de Carretera. Madrid.
- www.opengl.org
- G. A. Marcon dos Santos, V. T. Ferrão, C. Vinhal and G. da Cruz, "An adaptive algorithm for embedded real-time point cloud ground segmentation," *2015 7th International Conference of Soft Computing and Pattern Recognition (SoCPaR)*, Fukuoka, 2015, pp. 76-83.
- Pierre Merdrignac. Systeme cooperatif de perception et de communication pour la protection des usagers vulnérables. Automatique / Robotique. Ecole Nationale Supérieure des Mines de Paris, 2015.
- H. Zhu, P. Zhang, Y. Lin and J. Liu, "Joint detection and estimation fusion in distributed multiple sensor systems," *2016 19th International Conference on Information Fusion (FUSION)*, Heidelberg, 2016, pp. 805-810.
- H. Cho, Y. W. Seo, B. V. K. V. Kumar and R. R. Rajkumar, "A multi-sensor fusion system for moving object detection and tracking in urban driving environments," *2014 IEEE International Conference on Robotics and Automation (ICRA)*, Hong Kong, 2014, pp. 1836-1843.

Índice de Figuras

Figura 1: Línea de tiempo	13
Figura 2: Teleautomaton de Tesla	13
Figura 3: UAV	14
Figura 4: Vehículo autónomo de Google	15
Figura 5: Estructura de sensores en un vehículo autónomo	16
Figura 6: Sensores LIDAR	16
Figura 7: Visión sensor LIDAR	17
Figura 8: Cámara Mobileye	18
Figura 9: Visión de una cámara Mobileye	18
Figura 10: Cámara estéreo	19
Figura 11: Visión de una cámara estéreo	19
Figura 12: Radar	20
Figura 13: GPS	20
Figura 14: Unidad de Movimiento Inercial	21
Figura 15: Sensor de ultrasonidos	22
Figura 16: Ejemplo de planificación global de la trayectoria	25
Figura 17: Aproximación de trayectoria en una curva mediante curvas de Bézier	26
Figura 18: Aproximación de trayectoria en curvas consecutivas mediante curvas de Bézier	26
Figura 19: Evasión de obstáculos mediante campos potenciales	27
Figura 20: Aproximación de adelantamiento mediante lógica difusa	27
Figura 21: Niveles de autonomía de un vehículo autónomo	28
Figura 22: Coche autónomo Mercedes Benz	30
Figura 23: Coche autónomo Waymo	31
Figura 24: Visión del entorno de los vehículos Waymo	31
Figura 25: Coche autónomo Uber	31
Figura 26: Prototipo de vehículo autónomo Uber - Volvo	32
Figura 27: Coche autónomo Tesla	32
Figura 28: Percepción del entorno de un vehículo Tesla	33
Figura 29: Coche autónomo Baidu	34
Figura 30: Previsión de desarrollo de vehículos autónomos	35
Figura 31: Sensor LIDAR laser sick LRS 1000	40
Figura 32: Ejemplo de datos provenientes del laser	41
Figura 33: Arquitectura del sistema Fase 2	42
Figura 34: Diagrama UML sistema Fase 2	43
Figura 35: Velodyne VLP-16	44
Figura 36: Representación del entorno sensor VLP-16	45
Figura 37: Componente de entrada de datos RTMaps	46
Figura 38: Corrector de posición laser	46
Figura 39: Adquisición de datos Mobileye	47
Figura 40: Entrada de datos de los sensores	48
Figura 41: Vehículo C3 con el láser scanner montado	49
Figura 42: Implementación de la ecuación de tolerancia	52

Figura 43: Comprobar bloqueo	53
Figura 44: Implementación de la posible relación entre obstáculos	54
Figura 45: Intersección de trayectorias	55
Figura 46: Distancia de adelantamiento.....	56
Figura 47: Interfaz grafica.....	58
Figura 48: Inicialización de Arrays del controlador	59
Figura 49: Array de flags.....	60
Figura 50: Inicialización de objetos del controlador	61
Figura 51: Parámetros necesarios para el análisis	62
Figura 52: Constructor del análisis	63
Figura 53: Inicialización de parámetros.....	67
Figura 54: Núcleo del análisis	68
Figura 55: Comprobar bloqueo	68
Figura 56: Crear un nuevo obstáculo	69
Figura 57: Puntos a comprobar si son puntos cercanos.....	70
Figura 58: Código fuente de los puntos cercanos en horizontal.....	70
Figura 59: Código fuente de los puntos cercanos en diagonal	71
Figura 60: Código fuente de los puntos cercanos en vertical	71
Figura 61: Código en la función principal del filtrado de obstáculos	72
Figura 62: Código fuente de la función eliminar obstáculos.....	72
Figura 63: Código fuente del tracking de obstáculos	73
Figura 64: Relacionar por velocidad	74
Figura 65: Código fuente de la copia de obstáculos a la capa de control	75
Figura 66: Constructor del visualizador	76
Figura 67: Función trabajo del visualizador.....	76
Figura 68: Estructura básica de la ventana del visualizador	78
Figura 69: Código fuente de la función modificar puntos.....	79
Figura 70: Interfaz offline	80
Figura 71: Funcionamiento del pre proceso en un objeto de la cámara	84
Figura 72: Funcionamiento del pre proceso en un objeto del laser	84
Figura 73: Comparación de graficas del pre proceso	85
Figura 74: Calculo de la bounding box extendida 1	86
Figura 75: Calculo de la bounding box extendida 2	86
Figura 76: Calculo de la extended bounding box 3	87
Figura 77: Calculo de la extended bounding box 4	87
Figura 78: Resultado del primer paso de la asociación	88
Figura 79: Calculo de la puntuación de coincidencia de un parámetro.....	89
Figura 80: Matriz de posibles asociaciones y su puntuación	90
Figura 81: Código de búsqueda de ambigüedades	91
Figura 82: Resultado de la fusión	92

Índice de ecuaciones

Ecuación 1: Ecuación de tolerancia de distancia entre 2 puntos	52
Ecuación 2: Ecuación de reducción de ruido	52
Ecuación 3: Ecuaciones de la intersección de trayectorias	55
Ecuación 4: Ecuación de la distancia seguridad	56
Ecuación 5: Arquitectura básica Fase 2	57
Ecuación 6: Ecuación de posición respecto a la velocidad y tiempo	83
Ecuación 7: Calculo de la media mediante inferencia Bayesiana	92
Ecuación 8: Calculo de la varianza mediante inferencia Bayesiana	92
Ecuación 9: Fórmula de cálculo de la varianza modificada	93
Ecuación 10: Aclaración en la fórmula del cálculo de la varianza modificada	93